

Generics and Collections in Java 5

Maurice Naftalin
naftalin@morninglight.co.uk

Philip Wadler
wadler@inf.ed.ac.uk

*This version is a draft. Please send comments to the
addresses above.*

Copyright ©2005 O'Reilly Media, Inc.

Part III

Collections in Java 5

draft of 16th June 2005

Part I presented the features of Java 5 generics, often using generified collection classes as illustration. Collections aren't just a convenient way to illustrate how generics can be used, though—they are a primary motivation for having generics in the language at all. Virtually all working Java programs make heavy use of collections so it is a big benefit that, with the Java 5 generified collections, code that uses collections can be safe from type errors at runtime and also be clearer and more transparent to read. But the changes that make this possible take some work to understand and, together with the new interfaces and implementations and other changes in Java 5, justify reviewing all the features of the Java Collections Framework as a whole.

The Java Collections Framework (which we will often refer to as the Collections Framework, or just the Framework) is a set of interfaces and classes in the packages `java.util` and `java.util.concurrent`. They provide client programs with various models of how to organise their objects, and various implementations of each model. These models are sometimes called *abstract datatypes*, and we need them because different programs need different ways of organising their objects. In one situation you might want to organise your program's objects in a sequential list, because their ordering is important and there are duplicates. In another a set might be the right datatype, because now ordering is unimportant and you want to discard the duplicates. These two datatypes (and others) are represented by different interfaces in the Collections Framework, and we will look at examples of using them in this chapter. But that's not all; none of these datatypes has a single “best” implementation—that is, one implementation that is better than all the others for all the operations. For example, a linked list may be better than an array implementation of lists for inserting and removing elements from the middle, but much worse for random access. So choosing the right implementation for your program involves knowing how it will be used as well as what is available.

This chapter starts with an overview of the Framework, then looks in detail at each of the main interfaces, the standard implementations, and various special-purpose modifications of these. Finally we will look at the generic algorithms in the `Collections` class

Chapter 1

The Main Interfaces of the Java Collections Framework

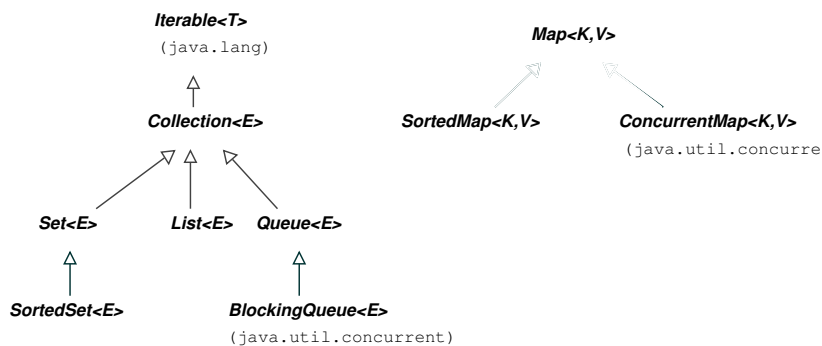


Figure 1.1: The main interfaces of the Java Collections Framework (extracted from back endpaper)

Figure 1 shows the main interfaces of the Java Collections Framework, together with one other—`Iterable`—which is outside the Framework but is an essential adjunct to it. It has the following purpose:

- `Iterable` defines the contract that a class has to fulfil for its instances to be usable with the *foreach* statement

and the Framework interfaces have the following purposes:

- `Collection` contains the core functionality required of any non-keyed collection. It has no direct concrete implementations; the concrete collection classes

all implement one of its subinterfaces as well

- **Set** is an unordered collection, without duplicates. **SortedSet** automatically sorts its elements and returns them in order
- **Queue** is a collection providing last-in-first-out element access. Processes calling methods on the **BlockingQueue** interface can be suspended until the requested operation can be carried out
- **List** is an ordered collection, accomodating duplicate elements
- **Map** is a collection which uses key-value associations to store and retrieve elements. In addition, **SortedMap** guarantees to return its values in ascending key order and **ConcurrentMap** provides methods that will be performed atomically

Chapters 3 to 7 will concentrate on each of the Collection Framework interfaces in turn. First, though, in chapter 2 we need to cover some preliminary ideas which run through the entire Framework design.

Chapter 2

Preliminaries

In this chapter we will take time to discuss the concepts underlying the framework before we get into the detail of the collections themselves.

2.1 Iterable and Iterators

An iterator is an object that implements the interface `Iterator` (Figure 2.1).

<i>Iterator<E></i>
<i>+hasNext() : boolean</i>
<i>+next() : E</i>
<i>+remove() : void</i>

Figure 2.1: `Iterator`

The purpose of iterators is to provide a uniform way of accessing collection elements sequentially. So whatever kind of collection you are dealing with, and however it is implemented, you always know how to process its elements in turn. For example, the following code for printing the string representation of a collection's contents will always work:

```
// coll refers to an object which implements Collection
// ----- not the preferred idiom in JDK1.5! -----
Iterator itr = coll.iterator() ;
while( itr.hasNext() ) {
    System.out.println( itr.next().toString() );
}
```

This works because any class implementing `Collection` will have an `iterator` method which returns an iterator appropriate to objects of that class. It's no

longer the approved idiom because Java 5 introduced something better: the *foreach* statement which you met in Part I. Using *foreach*, we can write the code above more concisely

```
for( Object o : coll ) {
    System.out.println( o );
}
```

and, what's more, this will work with anything that can give an iterator, not just collections. Formally, that means any object which implements the interface **Iterable** (Figure 2.2).

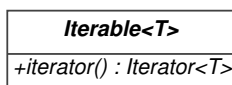


Figure 2.2: Iterable

Since in Java 5 the **Collection** interface has been made to extend **Iterable**, any collection can be the target of *foreach*, as can arrays. If you write your own implementation of **Iterable**, that too can be used with *foreach*. Figure 2.1 shows just about the simplest possible example of how **Iterable** can be directly implemented. A **Counter** object is initialised with a count of **Integer** objects; its iterator returns these in ascending order in response to calls of **next()**.

```
class Counter implements Iterable<Integer> {
    private int count;
    public Counter( int count ) { this.count = count; }
    public Iterator<Integer> iterator() {
        return new Iterator<Integer>() {
            private int i = 0;
            public boolean hasNext() { return i < count; }
            public Integer next() { i++; return i; }
            public void remove(){ throw new UnsupportedOperationException(); }
        };
    }
}
```

Figure 2.3: Directly implementing Iterable

Counter objects can now be the target of a *foreach* statement:

```
int total = 0;
```

```

for( Integer i : new Counter(3) ) {
    total += i;
}
assert total == 6;

```

In practice, it is unusual to implement `Iterable` directly in this way, as *foreach* is most commonly used with arrays and the standard collections classes.

2.2 Comparable and Comparator

One common requirement of collections is that they should be able to sort their contents according to some defined order. So a mechanism is needed for defining sort order for objects of any given class. In fact Java provides two mechanisms: in the first, a class can define a “natural order”, which an object of that class can use to compare itself with another. A collection can rely on the natural order or it can impose its own, overriding the default natural order.

To establish a natural order, a class implements the interface `Comparable` (Figure 2.4).

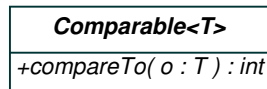


Figure 2.4: Comparable

According to the contract for `compareTo` the result should be a negative integer, zero, or a positive integer according to whether the supplied object is greater than, equal to, or less than the object making the comparison. Objects belonging to classes which implement `Comparable`—including many core classes such as `String` and the wrapper classes—can be sorted “automatically” by a collection or, as in this case, a utility method:

```

Integer[] ai = { 3, 2, 1, 4 };
Arrays.sort( ai );
assert Arrays.toString( ai ).equals( "[1, 2, 3, 4]" );

```

The alternative to allowing the set members to compare themselves is to provide an external means of comparing them. That means is usually an object implementing the interface `Comparator` (Figure 2.5).

Comparator<T>
<i>+compare(o1 : T, o2 : T) : int</i>
<i>+equals(obj : Object) : boolean</i>

Figure 2.5: Comparator

The `compare` method returns a negative integer, zero, or a positive integer according to whether its first argument is less than, equal to, or greater than its second. (The `equals` method is defined on this interface only so that its documentation can record that the motivation for overriding it is that clients may need to determine quickly if two different `Comparators` are equal—that is, if they impose the same order). Here is an example of declaring and using a `Comparator` that does not override `equals`:

```
Comparator<Integer> revComp =
    new Comparator<Integer>(){
        public int compare(Integer o1, Integer o2) {
            return (o1 < o2) ? 1 : (o1 > o2) ? -1 : 0;
        }
    };
Integer[] ai = { 3, 2, 1, 4 };
Arrays.sort( ai, revComp );
assert Arrays.toString( ai ).equals( "[4, 3, 2, 1]" );
```

The effect of the last example can be achieved more easily using the supplied method `reverseOrder` in the `Collections` class, which we shall look at in more detail later in this Part. (Don't confuse the convenience class `Collections` with the interface `Collection`).
[if that wasn't already said in part I, where the `Collections` and `Collection` are first seen] The method `reverseOrder` has two overloads:

```
static <T> Comparator<T> reverseOrder()
static <T> Comparator<T> reverseOrder(Comparator<T> cmp)
```

The first of these returns a `Comparator` which, applied to two objects, returns the reverse of what the natural order would produce. The second returns a `Comparator` with the reverse effect of the one supplied to it. So both the following `assert` statements will hold:

```
assert Arrays.sort( ai, revComp ).equals(
    Arrays.sort( ai, Collections.reverseOrder() ));

Comparator natural = Collections.reverseOrder( revComp );
assert Arrays.sort( ai ).equals( Arrays.sort( ai, natural ) );
```

Besides their use in the generic methods of `Collections` and `Arrays`, `Comparable` and `Comparator` have an important role in the construction of the

sorted collections `SortedSet` and `SortedMap`. These collections maintain their elements in order, as determined by their `Comparator` if one is supplied or otherwise by natural ordering. One aspect of this situation that may sometimes come as a surprise is that instead of using `equals` to determine equality between their elements, these collections rely on the `compare` or `compareTo` method they are using to return a value of 0 for equal elements. To avoid these (unpleasant) surprises, define `compare` or `compareTo` to be consistent with `equals`—to return 0 in exactly those cases that `equals` returns true. The discussion of `SortedSet` (Section 4.2) gives an example of what can happen otherwise.

2.3 Implementations

We have looked briefly at the interfaces of the Collections Framework, which define the behaviour that we can expect of each collection. But as we mentioned in the introduction to this chapter, there are several ways of implementing each of these interfaces. Why doesn't the Framework just use the best implementation for each interface? That would certainly make life simpler—too simple, in fact to be anything like life really is. If an implementation is a greyhound for some operations, Murphy's Law tells us that it will be a tortoise for others. Because there is no "best" implementation of any of the interfaces, you have to make a tradeoff—judging which operations are used most frequently in your application and choosing the implementation which optimises those operations.

The three main kinds of operations that most collection interfaces require are: insertion and removal of elements by position, retrieval of elements by content, and iteration over the collection elements. A glance at the back endpaper shows that there are many variations on these operations, but the main differences between different implementations can be discussed in terms of how they implement these three. In this section we'll briefly survey the four main kinds of implementation, and later when we need each one we will look at it in more detail. The four kinds are

- Arrays: these are the structures familiar from the Java language—and just about every other programming language since Fortran. Because arrays are implemented directly in hardware, they have the properties of random-access memory: very fast for accessing elements by position and for iterating over them, but slower for inserting and removing elements at arbitrary positions (because that may require adjusting the position of other elements). Arrays are used in the Collections Framework as the backing structure for `ArrayList`, `CopyOnWriteArraySet` and `CopyOnWriteArrayList`, and in most of the `Queue` implementations.
- Linked lists: as the name implies, these consist of chains of linked cells. Each cell contains a reference to data and a reference to the next cell in the list (and, in some implementations, the previous cell also). Linked lists perform

quite differently from arrays: accessing elements by position is slow, because you have to follow the reference chain from the start of the list, but insertion and removal operations can be performed in constant time by rearranging the cell references. Linked lists are used in `HashSet`, `LinkedHashSet`, and in the class `LinkedList`, which implements both `Queue` and `List`.

- Hash tables provide a way of storing elements indexed on their content rather than on an integer-valued index as with lists. In contrast to arrays and linked lists, hash tables provide no support for accessing elements by position, but access by content and insertion and removal is usually very fast. The first time that we will meet hash tables in use is in implementing the `Set` interface, in the class `HashSet` (Section 4.1.1).
- Trees also organise their elements by content, but with the important difference that they can store and retrieve them in sorted order. They are relatively fast for the operations of inserting and removing elements, accessing them by content, and iterating over them. We will first see trees in use as an implementation of the `SortedSet` interface (a subinterface of `Set`), in the class `TreeSet` (Section 4.2.1).

Besides allowing a focus on performance, another advantage to looking at implementations separately is that it will help you to understand the constructors of the different collection classes. We will see in section 3.2 that there are two standard kinds of constructor which almost all the collection classes have. But many also have other constructors, which are quite similar across implementations regardless of the interface being implemented. Since these are essentially for the purpose of configuring the implementations, we will discuss them along with other aspects of the implementations.

2.4 Efficiency and the O -notation

In the last section, we talked about different implementations being “good” for different operations. A good algorithm is economical in its use of two resources: time and space. Of the two, we are usually more interested in how fast an algorithm is than we are in the amount of memory it uses. It’s very hard to say precisely how quickly a program will execute, as that depends on many factors—including some that are outside the province of the programmer, such as the quality of the compiled code and the speed of the hardware. Even if we ignore these and limit ourselves to thinking only about how the execution time for an algorithm depends on its data, detailed analysis can be complex. A relatively simple example is provided in Knuth’s classic book *Sorting and Searching* (Addison-Wesley, 1973), where the worst-case execution time for a multi-list insertion sort is given as

$$3.5N^2 + 24.5N + 4M + 2$$

where N is the number of elements being sorted and M is the number of lists.

As a shorthand way of describing algorithm efficiency, this isn't very convenient. Clearly we need a broader brush for general use; the one most commonly used is the *O-notation* (pronounced as the “big-oh notation”). The *O-notation* describes how an algorithm performs “in the limit”, in other words when its data set becomes really large. In the expression above, the first two terms are comparable for low values of N ; indeed, for $N < 8$ the second term is larger. But as N grows the first term increasingly dominates the expression, and by the time it reaches 100 the first term is 15 times as large as the second one. Using a very broad brush, we say that the worst-case for multi-list insertion sort takes time $O(N^2)$. We don't care too much about the coefficient because it doesn't make any difference to the single most important question we want to ask about any algorithm: what happens to the running time when the data size increases, say when it doubles? For the worst-case insertion sort, the answer is that the running time goes up fourfold. That makes $O(N^2)$ pretty bad, worse than most algorithms in common use and worse than any we will meet in practical use in this chapter.

Figure 2.6 shows some commonly-found running times, together with examples of algorithms to which they apply. Many other running times are possible, including some that are much worse than those in the table. Many important problems can only be solved by algorithms that take $O(2^N)$ —for these, when N doubles the running time is squared! For all but the smallest data sets, such algorithms are infeasibly slow.

$O(\dots)$	commonly called	effect on the running time if N is doubled	example algorithms
1	constant	unchanged	insertion into a hash table (Section 4.1)
$\log N$	logarithmic	increases by a constant amount	insertion into a tree (Section 4.2.1)
N	linear	doubled	linear search
$N \log N$		doubled plus a constant amount	merge sort (Section 9.1)
N^2	quadratic	increases fourfold	bubble sort

Figure 2.6: Some Common Running Times

Sometimes we have to think about situations in which the cost of an operation varies with the state of the data structure; for example the normally-inexpensive operation of adding an element to a tree will sometimes trigger the need for a relatively expensive tree rebalancing operation. In these situations we calculate the *amortized cost* of the operation—that is, the total cost of performing it n times (where n is large) divided by n .

2.5 Contracts

In reading about software design, you are likely to come across the term *contract*, often without any accompanying explanation. In fact, software engineering gives this term a meaning very close to what people usually understand a contract to be. In everyday usage, a contract defines what two parties can expect of each other, their obligations to each other, in some transaction. If a contract specifies the service that a supplier is offering to a client, the supplier's obligations are obvious. But the client too may have obligations—besides the obligation to pay—and failing to meet them will automatically release the supplier from her obligations too. For example, airlines' conditions of carriage—for our class of tickets anyway—release the airline from the obligation to carry a passenger who has failed to turn up in time. This allows the airline to plan their service on the assumption that all the passengers they are carrying were punctual; they do not have to incur extra work to accommodate clients who have not fulfilled their side of the contract.

Contracts work just the same way in software. If the contract for a method states preconditions on its arguments (*i.e.* the obligations that a client must fulfil) the method is only required to return its contracted results when those preconditions are fulfilled. For example, binary search (Section 9.3) is a fast algorithm to find a key within an ordered list, and it fails if you apply it to an unordered list. So the contract for `Collections.binarySearch` can say “if the list is unsorted, the results are undefined”, and the implementer of binary search is free to write code which given an unordered list returns random results, throws an exception, or even enters an infinite loop. In practice, this situation is relatively rare in the contracts of the core API, because instead of restricting input validity they mostly allow for error states in the preconditions and specify the exceptions that the method must throw if it gets bad input. We think this is a practice to avoid, because it restricts the flexibility of the supplier unnecessarily. All that a client should need to know is how to keep to its side of the contract; if it fails to do that, all bets are off and there should be no need to say exactly what the supplier will do.

It's good practice in Java to code to an interface rather than to a particular implementation, so as to provide the maximum flexibility in choosing implementations. For that to work, what does it imply about the behaviour of implementations? If your client code uses methods of the `List` interface, for example, and at runtime the object doing the work is actually an `ArrayList`, you need to know that the assumptions you have made about how `Lists` behave are true for `ArrayLists` also. So a class implementing an interface has to fulfil all the obligations laid down by the terms of the interface contract. (Of course, a weaker form of these obligations is already imposed by the compiler; a class claiming to implement an interface must provide concrete method definitions matching the declarations in the interface; contracts take this further by specifying the behaviour of these methods as well).

The Collections Framework separates interface and implementation obligations in an unusual way. Some API methods return collections with restricted functionality—for example, the set of keys that you can obtain from a `Map` can have

elements removed but not added (Chapter 7). Others can neither have elements added or removed (for example the list view returned by `Arrays.asList()`), or may be completely read-only, like collections which have been wrapped in an unmodifiable wrapper (Section 8.2.1). To accommodate this variety of behaviours in the Framework without an explosion in the number of classes and interfaces, the designers labelled the modification methods in the `Collection` interface (and in the `Iterator` and `ListIterator` interfaces as well) as *optional operations*. If a client tries to modify a collection using an optional operation which the collection does not implement, the method must throw `UnsupportedOperationException`. The drawback to this approach is that a client programmer can no longer rely on the contract for the interface, but has to know which implementation is being used and to consult the contract for that as well. That's such a serious drawback that you will probably never be justified in subverting interfaces like this in your own designs. [Add discussion of relationship between contract and tests.]

2.6 Collections and Thread-Safety

A Java program when running is executing one or more execution streams, or *threads*. A thread is like a lightweight process, so a program simultaneously executing several threads can be thought of like a computer running several programs simultaneously, but with one important difference—different threads can simultaneously access the same memory locations and other system resources. On machines with multiple processors truly concurrent thread execution can be achieved by assigning a processor to each thread. If instead there are more threads than processors—the usual case—multithreading is implemented by *time-slicing*, in which a processor executes some instructions from each thread in turn before switching to the next one. Time-slicing is often fast enough to give the illusion that each thread has its own processor, and in fact this is the best way to imagine what is happening when you are programming for multiple threads.

There are two good reasons for using multithread programming: an obvious one, in the case of multiprocessor machines, is to share the work out and get it done quicker. (This reason will become even more compelling in the next few years as chip designers turn increasingly to architectures which depend on running multiple threads on even a single processor). The second reason is that many tasks can best be thought of as separate execution streams. A good example is the garbage collector, which runs in the background in every JVM. On a single-processor machine, the garbage collector gets its share of processor cycles via time-slicing, but that's irrelevant detail to the programmer writing a garbage collector—she should think of it as an independent autonomous process running in parallel with the other threads executing on the same JVM. Using that model will simplify her task enormously.

Inevitably, the conceptual simplification that threads can bring comes at a price. Different threads simultaneously accessing the same memory location can produce

some very unexpected results, unless you take care to constrain their access. For a very simple example, consider a program which uses a stack, modelled by an array `arr` and an `int` variable `i` which points at the top element of the stack. For the program to work correctly, it should ensure that however many elements are added to or removed from the stack, `i` should always point at the top element. This is called an *invariant* of the program. If both `arr` and `i` are variables of the same class, it is an invariant of that class.

Now think about what can happen if two threads simultaneously attempt to add an element to the stack. Each will execute the same code, say

```
arr[i] = ... ; //1
i++ ;         //2
```

which is correct in a single-threaded environment but may break the invariant in some multi-thread execution sequences. For example, if thread A executes line //1, thread B executes line //1 and then line //2, and finally thread A executes line //2, the overall result will have been to add one element to the stack, but to increment the stack pointer by two. The program is now in an inconsistent state, and is likely to fail because other parts of it will depend on the invariant being true. Code like this, that maintains program consistency when executed by a single thread but may break it in a multi-threaded environment, is not *thread-safe*. By contrast it would be possible to code this program so that once thread A had started to execute line //1, all other threads were excluded until it had successfully executed line //2. This is called *synchronizing* on a *critical section* of code. The synchronized code would be thread-safe, but at a price: forcing threads to queue up to enter the critical section one at a time will slow down the overall execution of the program. Even in a single-threaded environment synchronization has a performance cost.

This tradeoff between performance and thread-safety has been viewed differently at different times during the development of the Java collection classes. In versions before Java 2 the only collection classes provided were `Vector`, `Hashtable` and their subclasses, now regarded as legacy classes to be avoided. They were written to be thread-safe (by synchronizing most of their methods) but the resulting poor performance led the Java 2 collection designers to avoid internal synchronization in most of the classes that now constitute the Framework and which we will be discussing in this chapter.

The exceptions to this rule are the classes in `java.util.concurrent` (Sections 5.3 and 7.4) and objects produced by the synchronization wrappers described in Chapter 8. Together with the legacy collections classes, these fall into a category sometimes described as *conditionally thread-safe* (Bloch, *Effective Java*): they are thread-safe by the definition above, but sequences of operations on them may still require external synchronization. For example, the operation of adding an element to a collection if it is currently absent can only be made atomic (and therefore consistently correct) by synchronizing the code sequence which first tests for its presence and then adds it if appropriate. Only one collection class, `ConcurrentHashMap` (Section 7.4), has atomically-executed methods like `putIfAbsent` which makes it

(unconditionally) thread-safe.

2.6.1 Thread Safety and Iterators

If a method call breaks the conditions of the contract, the method is *entitled* to do anything. But to help the client programmer understand the problem, it would be *helpful* to fail in an indicative way. This is especially true when concurrency is involved, as otherwise failure may only become apparent some time after thread interference has occurred, making the problem difficult to diagnose. The non-thread-safe collection classes provide a helpful mechanism which can give clear and early warning of concurrent access. This mechanism is built into the iterators; if an iterator is described as *fail-fast* in the documentation, its methods will check that the collection has not been structurally changed (broadly speaking, that elements have been added or removed) since the last iterator method call. If it detects a change it will throw a `ConcurrentModificationException`. Of course, this mechanism cannot trap all concurrent access, but it will catch the most common cases.

You may have noticed that this description allows for “concurrent” modifications to be detected even in a single-threaded environment. Any structural changes to a collection during the lifetime of its iterator can result in a `ConcurrentModificationException`. This restriction rules out some sound programs (see, for example, Section 6.1), but many more unsound ones.

Chapter 3

The Collection Interface

`Collection` (Figure 3.1) defines the core functionality that we expect of any non-keyed collection. It provides methods in four groups:

Adding elements:

```
boolean add( E o )                // add the element o
boolean addAll( Collection<? extends E> c ) // add the contents of c
```

The boolean result returned by these methods indicates whether the collection was changed by the call. It can be false for collections, like sets, which will be unchanged if they are asked to add an element which is already present. But the method contracts specify that the elements being added must be present after execution, so if the collection refuses an element for any other reason (for example, some collections can't have `null` elements) these methods must throw an exception.

The signatures of these methods show that, as one might expect, you can only ever add elements or element collections of the parametric type.

Removing elements:

```
boolean remove( Object o )        // remove the element o
```

If the element `o` is null, `remove` removes a null from the collection if one is present. Otherwise if there is an element `e` present for which `o.equals(e)` it removes it. If not, it leaves the collection unchanged.

```
void clear()                      // remove all elements
boolean removeAll( Collection<?> c ) // remove the elements in c
boolean retainAll( Collection<?> c ) // remove the elements *not* in c
```

Where a method in this group returns a boolean function, it is true if the collection changed as a result of applying the operation.

In contrast to the methods for adding elements, these methods—and those of the next group—will accept elements or element collections of any type. We

Collection<E>
<pre> +add(o : E) : boolean +addAll(c : Collection<? extends E>) : boolean +clear() : void +contains(o : Object) : boolean +containsAll(c : Collection<?>) : boolean +equals(o : Object) : boolean +hashCode() : int +isEmpty() : boolean +iterator() : Iterator<E> +remove(o : Object) : boolean +retainAll(c : Collection<?>) : boolean +removeAll(c : Collection<?>) : boolean +size() : int +toArray() : Object[] +<T>toArray(T[] a) : T[] </pre>

Figure 3.1: Collection

will explain this in a moment, when we look at examples of the use of these methods.

Querying the contents of a collection:

```

boolean contains(Object o)           // true if o is present
boolean containsAll(Collection<?> c) // true if all elements of c are present
boolean isEmpty()                    // true if any elements are present
int size()                           // returns the element count (or
                                     // Integer.MAX_VALUE if that is less)

```

Making a collection's contents available for further processing:

```

Iterator iterator()                 // returns an Iterator over the elements
Object[] toArray()                  // copies contents to an Object[]
<T> T[] toArray( T[] t )           // copies contents to a T[] (for any T)

```

The last two methods in this group convert collections into arrays. Arrays still have some advantages over collections, and many APIs, especially older ones which predate the Java Collections Framework, have methods with array type parameters. The first method will create a new array of type `Object`. Given that this is a method of `Collection<E>` you might have hoped that it would have a return type of `E[]`. But recall from Part I that arrays carry their type information with them. To create an array of `E`, the system would have to know at run-time what `E` is, when in fact that information was discarded at compile-time by the process of erasure.

The second overload of `toArray()` allows the programmer to provide a reference to any array type `T[]`. At run time, the elements of the collection are placed in the array if there is room for them; otherwise a new array of that type is created. The first case can be useful if you want to allow the `toArray` method to reuse

an array that you supply; this can be more efficient, particularly if the method is being called repeatedly. The second case is more convenient—a common and straightforward idiom is to supply an array of zero length:

```
Collection<String> cs = ...
String[] sa = cs.toArray( new String[0] );
```

Even though the supplied array will not usually have sufficient room to contain the elements of the collection, it still fulfils the important function of telling the runtime system what type to give to the newly created array. But why is *any* type allowed for T? Actually there is a constraint that we would like to impose, but there is no way to write it. Suppose that you knew that all elements of a collection belonged to a particular subtype. Then it might make sense to transfer them all to an array of that subtype:

```
Collection<Number> cn = ...
Collection<Integer> ci = ...
cn.retainAll( ci );
Integer[] ai = cn.toArray( new Integer[] );
```

so in some circumstances you might want to use this method supplying an array of a subtype of the parametric type. But that won't always be so; in other circumstances it might be convenient to transfer all the elements of a subtype collection to a supertype array:

```
Number[] ta = ci.toArray( new Number[] );
```

To accommodate both possibilities, there has to be no constraint on T at all.

3.1 Using the Methods of Collection

To illustrate the use of the collection classes, let's construct a tiny example. The two of us are forever trying to get organised; let's imagine that our latest effort involves writing our own to-do manager. A central class in this system will be the **Task**. Parts of the information that constitutes a **Task** are its priority and a **String** describing it. The natural ordering of **Tasks**, as defined by `compareTo`, is the natural ordering of their string representations. We have chosen string representations for the **Task** subclasses which mean that coding tasks will always be placed before telephoning tasks of the same priority—we are geeks, after all.

```
enum Priority { HIGH, MEDIUM, LOW };

class Task implements Comparable<Task> {
    private Priority priority;
    protected String details;
    Task( String details, Priority priority ) {
        this.details = details;
```

```

        this.priority = priority;
    }
    public int compareTo( Task t ) {
        return toString().compareTo( t.toString() );
    }
    Priority getPriority() {
        return priority;
    }
    // definition of equals() and hashCode() omitted
}

```

We have divided the tasks to be scheduled into various subclasses:

```

class PhoneTask extends Task {
    String number;
    PhoneTask( String name, Priority priority, String number ) {
        super( name, priority );
        this.number = number;
    }
    public String toString() {
        return "ph. " + details;
    }
    // definition of equals() and hashCode() omitted
}
class CodingTask extends Task {
    CodingTask( String specReference, Priority priority ) {
        super( specReference, priority );
    }
    public String toString() {
        return "code " + details;
    }
    // definition of equals() and hashCode() omitted
}

```

Now we can define (Figure 3.2) a series of tasks that to be carried out (even if in a real system they would be more likely to be retrieved from a database)

```

PhoneTask nickPT = new PhoneTask( "Nick", Priority.MEDIUM, "987 6543" );
PhoneTask philPT = new PhoneTask( "Phil", Priority.HIGH, "123 4567" );
CodingTask blCT = new CodingTask( "bus. logic", Priority.MEDIUM );
CodingTask dbCT = new CodingTask( "db", Priority.HIGH );
CodingTask uiCT = new CodingTask( "ui", Priority.MEDIUM );

```

Figure 3.2: Example tasks for the Task Manager

For convenience, we're going to choose `ArrayList` as the implementation of `Collection` to use in this example, but we're not going to take advantage of any of the special properties of lists; we're treating `ArrayList` as an implementation of `Collection` and nothing more. As part of the retrieval process we can use the facilities provided by `Collection` to organise our tasks into various categories, using the method `Arrays.asList` discussed in Part I (ref??):

```
Collection<Task> urMondayTasks = Arrays.asList( philPT, dbCT );
Collection<Task> urTuesdayTasks = Arrays.asList( nickPT, blCT, uiCT );
Collection<PhoneTask> urPhoneTasks = Arrays.asList( nickPT, philPT );
Collection<CodingTask> urCodingTasks = Arrays.asList( blCT, dbCT, uiCT );
```

This isn't going to be quite good enough for our purposes, as `Arrays.asList` returns an unresizable `List`, as permitted by the definition of the optional operations of `Collection` (Section 2.5), and we want to use all the `Collection` operations on them. We have to relax our rule of not using any of the properties special to `ArrayList` in order make new resizable `Collections`.

```
Collection<Task> mondayTasks = new ArrayList( urMondayTasks );
Collection<Task> tuesdayTasks = new ArrayList( urTuesdayTasks );
Collection<PhoneTask> phoneTasks = new ArrayList( urPhoneTasks );
Collection<CodingTask> codingTasks = new ArrayList( urCodingTasks );
```

Although the constructor we are using here belongs to the `ArrayList` class and not to the interface, we aren't assuming too much, because in fact amongst the concrete classes which implement sets, queues and lists, all have a constructor which takes an argument of type `Collection`.

Now we can use the methods of `Collection` to work with these categories. The following examples use the methods in the order in which they were presented above.

- adding elements — we can add new tasks to the schedule:

```
mondayTasks.add( new PhoneTask( ... ) );
```

or we can combine schedules together:

```
Collection<Task> allTasks = new ArrayList<Task>( mondayTasks );
allTasks.addAll( tuesdayTasks );
```

- removing elements— when a task is completed we can remove it from a schedule:

```
boolean wasPresent = mondayTasks.remove( philPT );
```

and we can clear a schedule out altogether because all its tasks have been done (yeah, right):

```
mondayTasks.clear();
```

The removal methods also allow us to combine entire collections in various ways. For example, to see which tasks other than phone calls are scheduled for Tuesday, we can write:

```
Collection<Task> tuesdayNonphoneTasks =
    new ArrayList<Task>( tuesdayTasks );
tuesdayNonphoneTasks.removeAll( phoneTasks );
```

or to see which phone calls are scheduled for that day:

```
Collection<Task> tuesdayPhoneTasks =
    new ArrayList<Task>( tuesdayTasks );
tuesdayPhoneTasks.retainAll( phoneTasks );
```

This last example can be approached differently:

```
Collection<PhoneTask> tuesdayPhoneTasks =
    new ArrayList<PhoneTask>( phoneTasks );
tuesdayPhoneTasks.retainAll( tuesdayTasks );
```

This example provides an explanation of the signatures of methods in this group and the next. Why do they take arguments of type `Object` or `Collection<?>` when the methods for adding to the collection restrict their arguments to its parametric type? Taking the example of `retainAll`, its contract requires the removal of those elements of this collection which do not occur in the argument collection. That gives no justification for restricting what the argument collection may contain—in the example above it can contain instances of any kind of `Task`, not just `PhoneTask`. And it is too narrow even to restrict the argument to collections of supertypes of the parametric type; we want the least restrictive type possible, which is `Collection<?>`. Similar reasoning applies to `remove`, `removeAll`, `contains` and `containsAll`.

- querying the contents of a collection—these methods allow us to check, for example, that the operations above have worked correctly. We are using `assert` here to make the system check our belief that we have programmed the previous operations correctly. For example the first statement will throw an `AssertionError` if we had not succeeded in adding `nickPT` to `tuesdayPhoneTasks`:

```
assert tuesdayPhoneTasks.contains( nickPT );
assert tuesdayTasks.containsAll( tuesdayPhoneTasks );
assert mondayTasks.isEmpty();
assert mondayTasks.size() == 0;
```

- making the collection contents available for further processing — the methods in this group provide an iterator over the collection or convert it to an array. Section 2.1 showed how the simplest—and most common—explicit use of iterators has been replaced in Java 5 by the *foreach* statement, which uses them

implicitly. But there are uses of iteration with which *foreach* can't help. For example, supposing we want to merge two lists, of phone and of coding tasks, into a new list in which they are in their natural order. For little added effort we can generalise this task and merge any two collections, provided that iterators of each return their elements in their natural order:

```
static <T extends Comparable> List<T> mergeCollections(
    Collection<? extends T> c1, Collection<? extends T> c2 ) {

    List<T> mergedList = new ArrayList<T>();

    Iterator<? extends T> c1Itr = c1.iterator();
    Iterator<? extends T> c2Itr = c2.iterator();

    T c1Task = getNextTask( c1Itr );
    T c2Task = getNextTask( c2Itr );

    // each iteration will take a task from one of the iterators;
    // continue until neither iterator has any further tasks
    while( c1Task != null || c2Task != null ) {

        // use the current phone task if either the current coding
        // task is null or both are non-null and the phone task
        // precedes the coding task in the natural order
        boolean useC1Task = c2Task == null ||
            c1Task != null && c1Task.compareTo( c2Task ) < 0;

        if ( useC1Task ) {
            mergedList.add( c1Task );
            c1Task = getNextTask( c1Itr );
        } else {
            mergedList.add( c2Task );
            c2Task = getNextTask( c2Itr );
        }
    }
    return mergedList;
}

static <E> E getNextTask( Iterator<E> itr ) {
    return itr.hasNext() ? itr.next() : null ;
}
```

and now we can check whether this works:

```
assert mergeCollections( phoneTasks, codingTasks ).toString().equals (
    "[code bus. logic, code db, code ui, ph. Nick, ph. Phil]" );
```

3.2 Implementing Collection

The only direct implementation of the `Collection` interface in the Collections Framework is provided by the class `AbstractCollection`, which is the root of the hierarchies of sets, queues and lists. Figure 3.3 shows code functionally equivalent to the standard `AbstractCollection` but favouring conciseness over efficiency to show the essence of what an implementation must provide.

```

public abstract class AbstractCollection<E> implements Collection<E> {
    protected AbstractCollection() {}
    public abstract Iterator<E> iterator();
    public abstract int size();
    public boolean isEmpty() { return size() == 0; }
    public boolean contains( Object o ) {
        for ( E el : this ) {
            if ( o == null ? el == null : o.equals( el ) ) return true;
        }
        return false;
    }
    public Object[] toArray() { return toArray( new Object[size()] ); }
    public <T> T[] toArray(T[] a) {
        int size = size();
        if ( a.length < size ) {
            a = (T[])java.lang.reflect.Array.newInstance(
                a.getClass().getComponentType(), size );
        }
        int i = 0;
        for ( E el : this ) a[i++] = (T)el;
        return a;
    }
    public boolean add( E o ) { throw new UnsupportedOperationException(); }
    public boolean remove( Object o ) {
        Iterator<E> e = iterator();
        while( e.hasNext() ) {
            E el = e.next();
            if ( o == null ? el == null : o.equals( el ) ) {
                e.remove();
                return true;
            }
        }
        return false;
    }
}

```

```

public boolean containsAll( Collection<?> c ) {
    boolean result = true;
    for ( Object el : c ) { result = result && contains( el ); }
    return result;
}
public boolean addAll( Collection<? extends E> c ) {
    boolean modified = false;
    for ( E el : c ) { modified = add( el ) || modified; }
    return modified;
}
public boolean removeAll( Collection<?> c ) {
    boolean modified = false;
    for ( Object el : c ) { modified = remove( el ) || modified; }
    return modified;
}
public boolean retainAll( Collection<?> c ) {
    boolean modified = false;
    for ( E el : this ) {
        if ( ! c.contains( el ) ) {
            remove( el );
            modified = true;
        }
    }
    return modified;
}
public void clear() { for ( E el : this ) remove( el ); }
public String toString() {
    StringBuilder builder = new StringBuilder();
    builder.append( "[" );
    for ( E el : this ) {
        builder.append( String.valueOf( el ) + ", " );
    }
    builder.setCharAt( builder.length() -1, ']' );
    return builder.toString();
}
}

```

Figure 3.3: Outline implementation of AbstractCollection

The code in Figure 3.3 implements all but two of the methods inherited from `Collection`. These are `iterator` and `size`, and all the other methods of `Collection` are implemented in terms of these two. The implementation of `add` that is provided, however, simply throws an `UnsupportedOperationException`, so for collections that are not to be read-only it must be overridden as well. Most of the

methods of Figure 3.3 are straightforward and require no comment; the exception is `toArray(T[] t)`. Recall that the argument `t` is an array which the method can use for one of two purposes: it can accommodate the collection elements if it is long enough; if not, it can provide the type information needed to create a new array which will be long enough. It is this second alternative which is implemented by the line

```
a = (T[])java.lang.reflect.Array.newInstance(
    a.getClass().getComponentType(), size );
```

The method `newInstance` of `java.lang.reflect.Array` creates a new array given the component type and the number of elements required. Here the component type is being copied from the parameter, and the length of the new array is taken to be that of the collection to be stored in it. The casts in this method are unsafe, as you would expect given the discussion at the end of Section 3.1—any array can be provided as the argument to this method, so clearly there can be no compile-time guarantee of type-safety.

Note that this method omits one action of the standard implementation in the case that the supplied array is more than big enough to store the collection. In this situation the standard implementation places a `null` element immediately after the last stored collection element. This could be used as a sentinel to mark the end of the collection elements—but only, of course, if the method was being applied to a collection that does not accept `null` elements.

3.2.1 AbstractCollection subclasses

`AbstractCollection` is the root of the collections class hierarchy for sets, queues and lists. The relationship of `Collection` and `AbstractCollection` to `Set`, `Queue` and `List` and their abstract implementations is shown in Figure 3.4.

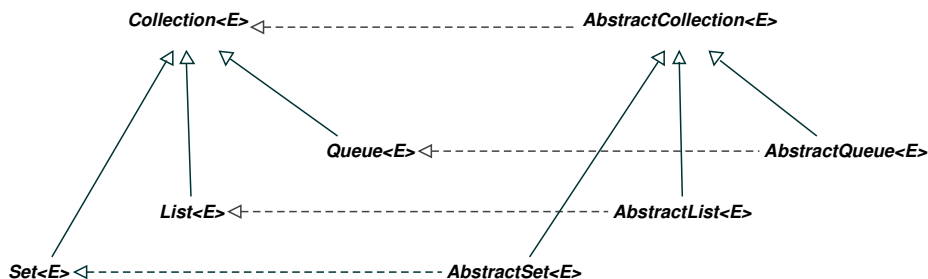


Figure 3.4: The place of `AbstractCollection` in the Java Collections Framework (extracted from the front endpaper)

Before we go on to look at these three main kinds of collection, which follow in the next three main sections, this is the moment for an explanation of the constructors that are available for **AbstractCollection** subclasses. The general rule for **Collection** implementations is that they have at least two constructors; taking as an example **HashSet** these are

```
public HashSet()  
public HashSet( Collection<? extends E> c )
```

The first of these creates an empty set, and the second a set that will contain the elements of any collection of the parametric type—or one of its subtypes, of course: using this constructor has the same effect as creating an empty set with the default constructor, then adding the contents of a collection using **addAll**. Not quite all collection classes follow this rule (**ArrayBlockingQueue** cannot be created without fixing on its capacity, and **SynchronousQueue** cannot hold any elements so no constructor of the second form is appropriate). In addition many collection classes have other constructors besides these two, but which ones they have depends not on the interface they implement but on the underlying implementation—these additional constructors are used to configure the implementation.

Chapter 4

Sets

A set is a collection of items which cannot contain duplicates—adding an item if it is already present in the set has no effect. The `Set` interface has the same methods as those of `Collection`, but is defined separately in order to allow the contract of `add` (and `addAll`, which is defined in terms of `add`) to be changed in this way. For example, suppose it is Monday and you have some free phone time, so that you have a chance to carry out all your telephone tasks. You can make the appropriate collection by adding all your telephone tasks to your Monday tasks. Using a set (again choosing a conveniently common implementation of `Set`) you can write

```
Set<Task> phoneAndMondayTasks = new HashSet<Task>( mondayTasks );
phoneAndMondayTasks.addAll( phoneTasks );
```

and this works because of the way that duplicate elements are handled. The task `nickPT`, which is in both `mondayTasks` and `phoneTasks`, only appears once in `phoneAndMondayTasks`, as intended—you definitely don't want to have to do all such tasks twice over!

4.1 Implementing Set

Section 3.1, which used the the methods of `Collection` in some examples, emphasised that they would work with any implementation of `Collection`. What if we had decided that we would use one of the `Set` implementations from the Collections Framework? We would have had to choose between the various concrete implementations that the Framework provides, which differ both in how fast they perform the basic operations of `add`, `contains` and iteration, and in the order in which their iterators return their elements. In this section we will look at these differences, then summarising the comparative performance of the different implementations in Section 4.3.

The only direct implementation of the `Set` interface in the Collections Framework is provided by the class `AbstractSet`, from which all the concrete implemen-

tations inherit. The contract for `AbstractSet` refines that of `AbstractCollection` in respect of what constitutes equality. Two sets are equal if and only if they contain the same elements—there is no dependence on ordering. Figure 4.1 shows an implementation of `AbstractSet`.

```

public abstract class AbstractSet<E> extends AbstractCollection<E>
                                   implements Set<E> {
    protected AbstractSet() { }
    public boolean equals(Object o) {
        if (!(o instanceof Set)) return false;
        return ((Set)o).size() == size() && containsAll( (Set)o );
    }
    public int hashCode() {
        int h = 0;
        for( E el : this ) {
            if ( el != null ) h += el.hashCode();
        }
        return h;
    }
}

```

Figure 4.1: Outline implementation of `AbstractSet`

The `equals` method of this class checks that the argument is itself a `Set` (a `Set` cannot be equal to any other kind of object), that the two `Sets` have the same size, and that one contains all the elements of the other. Provided that `size` fulfils its contract of returning the number of distinct elements that a `Set` contains, this guarantees that equal `Sets` must contain the same elements.

Let's see some working code: Figure 4.2 is a tiny implementation of `Set`. It provides implementations for the two methods — `size` and `iterator`—that are abstract in `AbstractSet`, and makes `add` into a supported operation, overriding the version inherited from `AbstractCollection`.

```

class FixedArraySet extends AbstractSet<E> {
    private final int MAX_SIZE = 10;
    private E[] array = (E[])new Object[MAX_SIZE];
    private int i = -1;
    public FixedArraySet() {}
    public FixedArraySet( Collection<? extends E> c ) { addAll(c); }
    public boolean add( E o ) {
        if ( ! contains( o ) ) {
            assert i < MAX_SIZE;
            array[i + 1] = o;
            i = i + 1;
            return true;
        } else {
            return false;
        }
    }
    public int size() { return i + 1 ; }
    public Iterator<E> iterator() {
        return new Iterator<E>() {
            private int j = -1;
            public boolean hasNext() { return j < i ; }
            public E next() { return array[++j]; }
            public void remove() {
                if ( j != i ) System.arraycopy( array, j + 1, array, j, i - j );
                i--; j--;
            }
        };
    }
}

```

Figure 4.2: Outline implementation of Set

This implementation is about the simplest you can imagine; being backed by a fixed array, it has a fixed maximum size. If assertions are enabled, attempting to add an element to a “full” `FixedArraySet` results in an `AssertionError`; otherwise an `ArrayIndexOutOfBoundsException` is thrown. This limitation makes `FixedArraySet` a toy implementation, as does its performance: `contains` has a complexity of $O(n)$ because it requires iteration through the set; since `add` calls `contains` its cost too is $O(n)$. These results are terrible compared to the standard implementations, as we shall see. The iteration operation itself, however, simply involves stepping through the backing array, so has complexity $O(1)$. This suggests that an array-backed implementation would in fact be a good choice for applications that depend on efficiently iterating through a set; `CopyOnWriteArraySet` is

provided for just this reason.

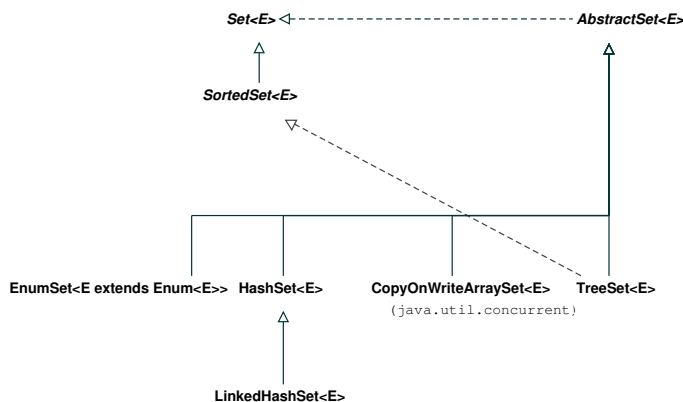


Figure 4.3: Implementations of the `Set` interface (extracted from the front endpaper)

There are five concrete implementations of `Set` in the Collections Framework. Figure 4.3 shows their relationship to `AbstractSet` and the `Set` subinterface `SortedSet`. In this section we will look at `HashSet`, `LinkedHashSet` and `CopyOnWriteArraySet`—`EnumSet` is specialised for use with enums and is discussed along with them in Section ??, and `SortedSet` is discussed together with its implementation `TreeSet` in Section 4.2.

4.1.1 HashSet

This class is the most commonly-used implementation of `Set`. As the name implies, it is implemented by a *hash table*, an array in which elements are stored at a position derived from their contents. Since hash tables store and retrieve elements by their content, they are well suited to implementing the operations of `Set` (the Collections Framework also uses them to for various implementations of `Map`). For example, to implement `contains(Object o)` you would want to look for the element `o` and return `true` if it were found.

The position of an element in a hash table is given by the *hash code* of its contents—a number calculated in a way that gives an even spread of results over the element values that might be stored. For example, the hash code of a string `str` is calculated as follows:

```

int hash = 0;
for ( char ch : str.toCharArray() ) {
    hash = hash * 31 + ch;
}

```

To get an index from the hash code we get its remainder after division by the table length. Since that means it is the pattern of bits at the low end of the hash code that is significant, prime numbers (like 31, here) are used in calculating the hash code because multiplying by primes will not tend to shift information away from the low end—as would multiplying by a power of 2, for example.

It's obvious that the algorithm above will give equal hash values for equal strings—essential if the same hash function is to be used for storing words as for retrieving them. But can it also give equal hash values for unequal strings? That would be a problem because it would imply that the hash table would have to store more than one element at each location. A moment's thought will show that in fact you can't avoid this problem completely without having an infinitely large array to hold the table. We can minimise the problem with a good hash function, one which spreads the elements out equally in the table, but when collisions do occur we have to have a way of keeping the colliding elements at the same table location or *bucket*. This is done by storing them in a linked list, as shown in Figure 4.4¹. We will look at linked lists in detail as one of the implementations of the `Queue` interface (Section 5.2) but for now it's enough to see that elements stored at the same bucket can still be accessed, at the cost of following a chain of cell references. In this example preserving the bottom three bits of the hash codes for the `Character` objects which box 'a' and 'b' give a bucket index of 2, while 'c' gives an index of 1.

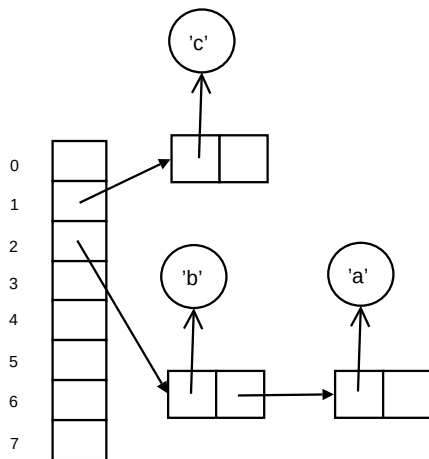


Figure 4.4: A hash table with chained overflow

As long as there are no collisions the cost of inserting or retrieving an element is constant. As the hash table fills, collisions become more likely; assuming a good hash function the probability of a collision in a lightly loaded table is proportional to its

¹This diagram is simplified from the real situation. Because `HashSet` is actually implemented by a specialised `HashMap`, each of the cells in the chain contains not one but two references, to a key and a value (see Section 7). Only the key is shown in this diagram.

load factor, defined as the number of elements in the table divided by its capacity (the number of buckets). If a collision does take place, a linked list has to be created and subsequently traversed, adding an extra cost to insertion proportional to the number of elements in the list. As the load factor continues to increase the performance worsens until eventually, usually at some predetermined load factor, it becomes worthwhile to copy the elements to a new and larger hash table.

Iterating over a hash table requires each bucket to be examined to see whether it is occupied or not and therefore costs a time proportional to the capacity of the hash table plus the number of elements it contains. Since the iterator examines each bucket in turn, the order in which elements are returned depends on their hash codes. The table shown in Figure 4.4 yields its elements in order of descending table index and forward traversal of the linked lists:

```
Set s1 = new HashSet(8);
s1.add('a');
s1.add('b');
s1.add('c');
assert s1.toString().equals("[b, a, c]");
```

Later in this section we will look at `LinkedHashSet`, a variant hash table implementation with an iterator which does return elements in their insertion order.

The chief attraction of a hash table implementation for sets is the constant-time performance (ideally) for the basic operations of `add`, `remove`, `contains` and `size`. Its main disadvantage is its iteration performance; since iterating through the table involves examining every bucket, its cost is proportional to the table size regardless of the size of the set it contains.

The Collections Framework class `HashSet` has some features which don't directly follow from this general description of how hash tables work. For one thing, `HashSet` delegates all its operations to a private instance of `HashMap` (Section 7.2) in which every set element is treated as a key with a corresponding standard value signifying that it is present. A second peculiarity follows from the first, because since JDK1.4 all instances of `HashMap` have a bucket count which is a power of two. So the constructors don't behave quite as you might expect with respect to the capacity. `HashSet` has the standard constructors that we introduced in Section 3.2.1, together with a further two:

```
HashSet( int initialCapacity )
HashSet( int initialCapacity, float loadFactor )
```

both of which create an empty set but allow some control over the size of the underlying table—creating one with a length of the next-largest power of 2 after the supplied capacity. The hash table-based implementations in the Collections Framework all have similar constructors.

4.1.2 CopyOnWriteArraySet

In functional terms `CopyOnWriteArraySet` is another straightforward implementation of the `Set` contract, but with quite different performance characteristics from `HashSet`. As the name implies, each instance of this class is essentially read-only; a change to the contents of the set results in an entirely new object being created. So `add` has complexity $O(n)$, as does `contains`, which has to be implemented by a linear search. Clearly you wouldn't use `CopyOnWriteArraySet` in a context where you were expecting many searches or insertions. But because this implementation is backed ultimately by an array, iteration over its elements is $O(1)$ —faster than `HashSet`—and it has one advantage which is really compelling in some applications: it provides thread safety (Section 2.6) at no additional cost. Normally, of course, thread-safe implementations are much less efficient because a thread must get exclusive access to a collection object before it can use it in any way. But if, as in the case of `CopyOnWriteArraySet`, such change is actually impossible, then thread-safety comes for free!

When would you want to use a set with these characteristics? A clue to the purpose of `CopyOnWriteArraySet` is given by its location in the package `java.util.concurrent`. It is a utility class which will be used mainly by classes implementing the Observer-Observable design pattern, which requires events to be notified to a set of observers. An awkward complication arises if this set is changed by another thread during the notification process, and the benefit of using `CopyOnWriteArraySet` is that it prevents that, and does so at no extra cost for read accesses.

Since there are no configuration parameters for a `CopyOnWriteArraySet`, the constructors are just the standard ones of Section 3.2.1.

4.1.3 LinkedHashSet

This class inherits from `HashSet`, still implementing `Set` and refining the contract of its superclass in only one respect: it guarantees that its iterators will return their elements in the order in which they were first added. It does this by maintaining a linked list of the set elements, as shown by the curved arrows in Figure 4.5. The resulting behaviour is just what you would expect:

```
Set s2 = new LinkedHashSet(8) {{ add('a'); add('b'); add('c'); }}; //1
assert s2.toString().equals("[a, b, c]");
```

The code at line //1 creates an anonymous local subclass of `HashSet` with an initializer which adds three elements on the creation of an object of the class. It's a concise (if somewhat tricky) way of simultaneously creating and initializing an object.

The linked structure also has a useful consequence in terms of improved performance: `next` performs in constant time, as the linked list can be used to visit each element in turn. This is in contrast to `HashSet`, for which every bucket in the hash table must be visited whether it is occupied or not, but the overhead involved in

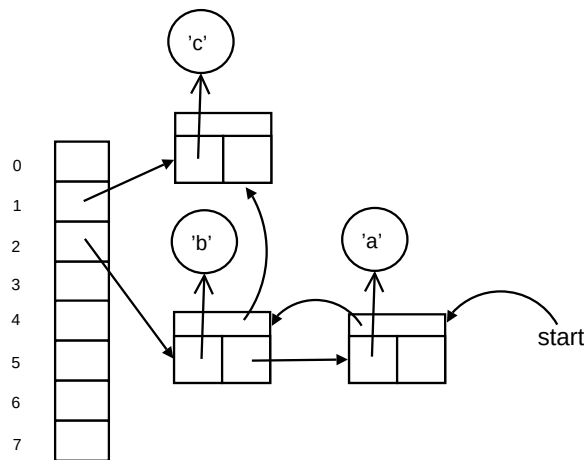


Figure 4.5: A linked hash table

maintaining the linked list means that you would only choose `LinkedHashSet` in preference to `HashSet` if the efficiency or the order of iteration were important for your application.

The constructors for `LinkedHashSet` provides the same facilities as those of `HashSet` for configuring the underlying hash table.

4.2 SortedSet

`Set` has one subinterface, `SortedSet` (see back endpaper) which adds to its contract a guarantee that its iterator will traverse the set in ascending element order. We discussed the different ways of defining element ordering earlier in this chapter: it can either be defined by the element class itself, if that implements `Comparable`, or it can be imposed by an external `Comparator`. `Task` implements `Comparable` (its natural ordering is the natural ordering of its string representation), so can be sorted “automatically”. Merging two ordered lists (Figure 3.2, page 24) which was quite tricky using parallel iterators (Section 3.1) is trivial if we use a `SortedSet` to do the work:

```
// TreeSet is the standard implementation of SortedSet
Set<Task> naturallyOrderedTasks = new TreeSet<Task>( phoneTasks );
naturallyOrderedTasks.addAll( codingTasks );
assert naturallyOrderedTasks.toString().equals (
    "[code bus, logic, code db, code ui, phone Nick, phone Phil]" );
```

The alternative to using the natural ordering is to supply the `SortedSet` with a `Comparator` at construction time. For example, another useful ordering for tasks would be in descending order of priority:


```

Comparator<Task> priorityComparator =
    new Comparator<Task>(){
        public int compare(Task o1, Task o2) {
            int p = o2.getPriority().compareTo( o1.getPriority() );
            return p != 0 ? p : o1.toString().compareTo( o2.toString() );
        }
    };
SortedSet<Task> priorityOrderedTasks = new TreeSet<Task>( priorityComparator );
priorityOrderedTasks.addAll( phoneTasks );
priorityOrderedTasks.addAll( codingTasks );
assert priorityOrderedTasks.toString().equals (
    "[code db, ph. Phil, code bus. logic, code ui, ph. Nick]" );

```

Could you not simply compare the priorities of the tasks, without using the string representation as a secondary key? You could, but you might not like the result. Recall that in the discussion of `Comparable` and `Comparator` (Section 2.2) we mentioned that the methods of these interfaces should be consistent with `equals`. The reason for this is that the contract for `SortedSet` (and, as we shall see later, `SortedMap` also) states that it will use the `compareTo` method of its `Comparator`—or, if it does not have one, the `compare` method of its elements—instead of the elements' `equals` method. This means that if two elements are equal in their ordering, the set will treat them as duplicates and one will be discarded.

The extra methods defined by the `SortedSet` interface fall into three groups:

Retrieving the comparator:

```
Comparator<? super E> comparator()
```

This method returns the set's comparator if it has been given one rather than relying on its elements to define their ordering. The type `Comparator<? super E>` is used because a `SortedSet` parameterized on `E` can rely for ordering on a `Comparator` defined on any supertype of `E`. For example, a `Comparator<Number>` could be used with a `SortedSet<Integer>`.

Getting the head and tail elements:

```
E first()
E last()
```

If the set is empty, these operations throw a `NoSuchElementException`.

Finding subsequences:

```
SortedSet<E> subset( E fromElement, E toElement )
SortedSet<E> headSet( E toElement )
SortedSet<E> subset( E fromElement )
```

The arguments to these operations do not themselves have to be members of the set. The sets returned are *half-open intervals*: they are inclusive of the `fromElement`—provided it really is a set member, of course—and exclusive of the `toElement`.

We could use these methods, for example, to obtain a view of the portion of `priorityOrderedSet` corresponding to the high-priority items by using `headSet`, providing as the argument the element which we wish to form the exclusive limit of the set — a medium-priority task defined so that it will come before any other in the secondary (description) ordering:

```
Task highestMediumPriorityTask = new Task( "\0", Priority.MEDIUM )
SortedSet<Task> highPriorityTasks =
    priorityOrderedTasks.headSet( highestMediumPriorityTask );
```

Notice that the sets returned by these operations are not newly created sets but different views of the original `SortedSet`. So changes in them will be reflected in the original `SortedSet`, and vice versa.

4.2.1 TreeSet

We have already met this class in discussing the interface `SortedSet`, for which it is the standard implementation and which completely specifies its contract. This is the first tree implementation that we have seen, so we should take a little time now to consider how trees perform in comparison to the other implementation types used by the Collections Framework.

Trees are the data structure you would choose for an application which needed fast insertion and retrieval of individual elements but which also required that they should be held in sorted order. For example, suppose you wanted to match all the words from a set against a given prefix, a common requirement in visual applications where a dropdown should ideally show all the possible elements that match against the prefix that the user has typed. A hash table can't return its elements in sorted order and a list can't retrieve its elements quickly by their content, but a tree can do both.

In computing, a tree is a branching structure that represents hierarchy. Computing trees borrow a lot of their terminology from genealogical trees, though there are some differences; the most important is that in computing trees each node has only one parent (except the root, which has none). An important class of tree often used in computing is a *binary* tree—one in which each node can have at most two children. Figure 4.6 shows an example of a binary tree containing the words of this sentence in alphabetical order.

The most important property of this tree can be seen if you look at any non-leaf node, say the one containing the word “the”: all the nodes below that on the left contain words that precede “the” alphabetically, and all those on the right words that follow it. To locate a word you would start at the root and descend level by level doing an alphabetic comparison at each level, so the cost of retrieving or inserting an element is proportional to the depth of the tree.

How deep then is a tree that contains n elements? The complete binary tree with two levels has 3 elements (that's $2^2 - 1$), the one with three levels has 7 elements ($2^3 - 1$) and so on. In general a binary tree with n complete levels will have $2^n - 1$

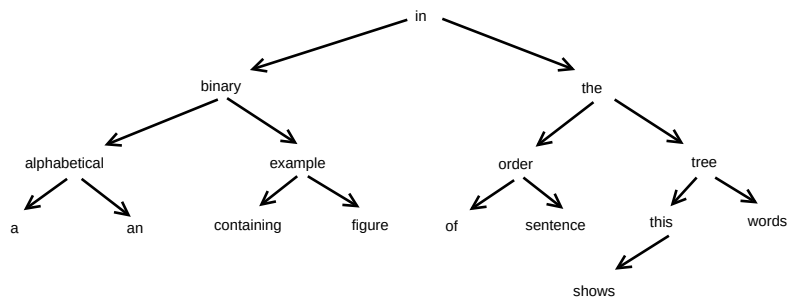


Figure 4.6: An ordered binary tree

elements. To work backwards from the number of elements to the number of levels you have to use a *log* function—a function with the property that $2^{\log(n)} = n$. All that you really need to know about log functions is that just as n grows much more slowly than 2^n , $\log(n)$ grows much more slowly than n . So **contains** on a large tree is much faster than on a list containing the same elements. It’s still not as good as on a hash table—whose operations can ideally work in constant time—but a tree has the big advantage over a hash table that its iterator can return its elements in sorted order.

Not all binary trees will have this nice performance, though. Figure 4.6 shows a *balanced* binary tree—one in which each node has an equal number of descendants (or as near as possible) on each side. An unbalanced tree can give much worse performance—in the worst case, as bad as a linked list (see Figure 4.7). **TreeSet** uses a data type called a *red-black tree*, which has the advantage that if it becomes unbalanced through insertion or removal of an element, it can always be rebalanced in $O(\log(n))$ time.

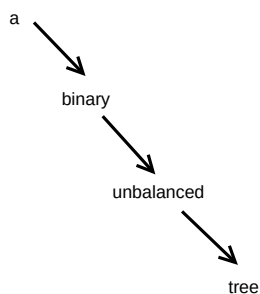


Figure 4.7: An unbalanced binary tree

The constructors for **TreeSet** include, beside the standard ones, one which allows you to supply a **Comparator** (Section 4) and one which allows you to create one

from another `SortedSet`, using both the same comparator and the same elements.

4.3 Comparison of Set Implementations

	<code>add</code>	<code>contains</code>	<code>next</code>	notes
<code>HashSet</code>	$O(1)$	$O(1)$	$O(n/h)$	h is the table capacity
<code>TreeSet</code>	$O(\log n)$	$O(\log n)$	$O(1)$	traversal in sorted order
<code>CopyOnWriteArraySet</code>	$O(n)$	$O(n)$	$O(1)$	thread-safe
<code>LinkedHashSet</code>	$O(1)$	$O(1)$	$O(1)$	traversal in inserted order

Chapter 5

Queues

A **Queue** (Figure 5.1) is a collection designed to hold elements to be processed, in the order in which processing is to take place. Different implementations of the interface embody different rules about what this order should be. Many of the implementations use the rule that tasks are to be processed in the order in which they were submitted (*First In First Out*, or *FIFO*), but other rules are possible—for example, the Collections Framework includes queue classes whose processing order is based on task priority.

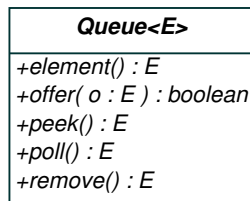


Figure 5.1: Queue

The **Queue** interface was introduced in Java 5, mainly motivated by the need for queues in the concurrency utilities included in that release. A glance at the hierarchy of implementations shown in the front endpaper figure shows that in fact nearly all of the **Queue** implementations in the Collections Framework are in the package `java.util.concurrent`.

One classic requirement for queues in concurrent systems comes when a number of tasks have to be executed by a number of threads working in parallel. An everyday example of this situation is that of a single queue of airline passengers being handled by a line of check-in operators. Each operator works on processing a single passenger (or a group of passengers) while the remaining passengers wait

in the queue. When a passenger reaches the head of the queue she waits to be assigned to the next operator to become free. There is a good deal of fine detail involved in implementing a queue like this; operators have to be prevented from simultaneously attempting to process the same passenger, empty queues have to be handled correctly, and in computer systems there has to be a way of defining queues with a maximum size, or *bound*. (This last requirement may not be at all desirable in airline terminals, but it can be very useful in systems in which there is a maximum reasonable waiting time for a task to be executed). The `Queue` implementations in `java.util.concurrent` look after these implementation details for you.

In addition to the operations inherited from `Collection`, `Queue` includes operations for:

Adding an element to a queue, if possible:

```
boolean offer(E o) // inserts the given element if possible
```

This operation may routinely fail, for example because the queue has reached its capacity. `offer` accommodates this failure by returning a boolean reporting whether the element was successfully inserted (in contrast to the `add` method of `Collection`, which can report failure only by throwing an exception).

Retrieving an element from a queue: The four methods provided for this purpose differ in two respects: whether they only provide a copy or also remove the element, and whether they return null or throw an exception in the case of an empty queue.

The methods which throw an exception for an empty queue are

```
E element() // retrieves but does not remove the head element
E remove() // retrieves and removes the head element
```

Notice that this is a different overload from the `Collection` method `remove(Object)`. The methods which return null for an empty queue are

```
E peek() // retrieves but does not remove the head element
E poll() // retrieves and removes the head element
```

Because of the use of null as a return value it is inadvisable to use it as a queue element; for this reason, most implementations do not allow it.

5.1 Using the Methods of Queue

Let's look at examples of the use of these methods. For us, the major goal of having a to-do manager is to tell us what we should do next—we find it altogether too much of a strain to work this out for ourselves. Queues are a good mechanism for storing this information. In simple cases, we may just decide that the best policy is to do tasks in the order in which we become aware of the need for them.

So new tasks should be added to the tail of the queue. For these examples we have chosen the `ConcurrentLinkedQueue`, which despite its unwieldy name is the most straightforward implementation.

```
Queue<Task> taskQueue = new ConcurrentLinkedQueue<Task>();
taskQueue.offer( nickPT );
taskQueue.offer( philPT );
```

Any time we feel ready to do a task we can take the one that has reached the head.

```
Task nextTask = taskQueue.poll();
if ( nextTask != null ) {
    // process nextTask
}
```

The choice between using `poll` and `remove` depends on whether we want to regard queue emptiness as an exceptional condition. Realistically—given the nature of this application—that might be a sensible assumption, so this could be preferable:

```
try {
    Task nextTask = taskQueue.remove();
    // process nextTask
} catch ( NoSuchElementException e ) {
    // but we *never* run out of tasks!
}
```

This scheme needs some refinement to allow for the nature of different kinds of tasks. Phone tasks can be fitted into relatively short time slots, whereas we don't like to start coding unless there is reasonably substantial time to get into the task. So if time is limited, say before the next meeting, we might like to check that the next task is of the right kind before we take it off the queue:

```
try {
    Task nextTask = taskQueue.element();
    if ( nextTask instanceof PhoneTask ) {
        // process nextTask
    }
} catch ( NoSuchElementException e ) { ... }
```

But notice that the `Queue` methods can't do much for us if the head element is not one that we want. Of course, the methods of `Collection` are still available, so we can get an iterator, but not all queue iterators guarantee to return their elements in proper sequence. So in this situation, if our to-do manager is entirely queue-based, it may be able to offer us no better alternative than to go for coffee until the meeting starts. We will see that the `List` interface can provide better facilities in this respect.

5.2 Implementing Queue

The concrete Collections Framework implementations of `Queue`, shown in Figure 5.2, differ widely in their behaviour. They are mostly, but not all, synchronized for use in multithreading programs; most are *blocking* queues, for which a subinterface `BlockingQueue` is provided; some support priority ordering; one—`DelayQueue`—holds elements until their delay has expired, and one—`SynchronousQueue`—is purely a synchronisation facility. In choosing between `Queue` implementations you would be influenced more by these functional differences than by their relative performances. Besides, a discussion of the efficiency of the concurrent implementations is outside the scope of this book.

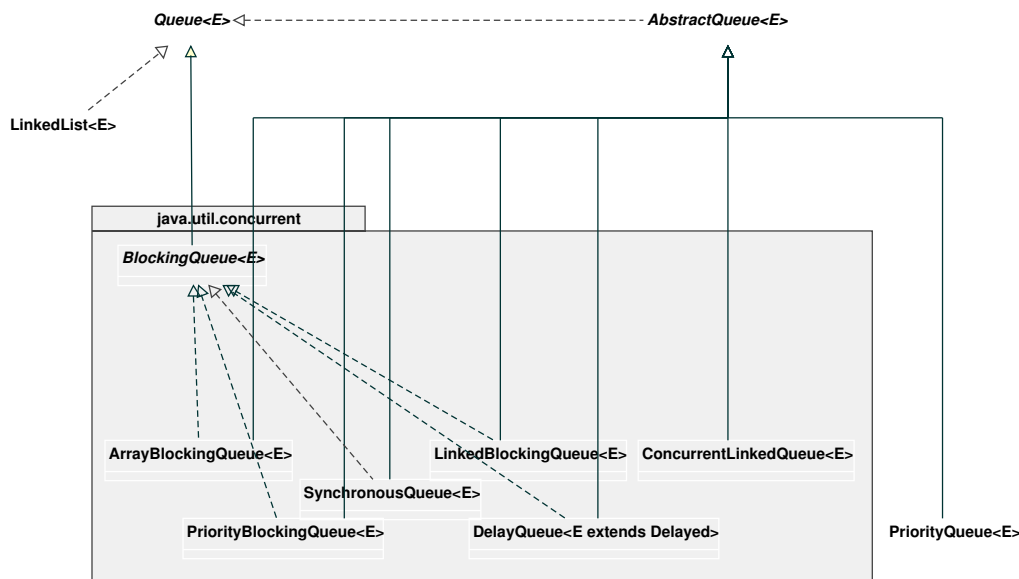


Figure 5.2: Implementations of `Queue` in the Collections Framework (extracted from front endpaper)

5.2.1 LinkedList

The first class that stands out in Figure 5.2 is `LinkedList`. This isn't specialised at all: it isn't thread-safe, nor does it support blocking or priorities. In fact, it was in the Collections Framework for a long time before queues were introduced, as one of the standard implementations of `List` (Section 6.2). It was retrofitted in Java 5 to implement `Queue` as well. In its simplest form, a linked list consists of a number of cells each with two fields, one to hold an element reference and one to allow an

iterator to reach the next cell. Each cell object would be an instance of a class defined like this:

```
class Cell<E> {
    private E element;
    private Cell next;
    ...
}
```

Figure 5.3(a) shows a simplified picture of a `LinkedList` after the following statement:

```
new LinkedList<Character>(){ add( nickPT ); add( philPT ); };
```

Removing an element from the list or inserting one into it is done by rearranging the cell references. For example, Figure 5.3(b) shows the effect of calling `remove` on this queue. A `LinkedList` also maintains a pointer to the last element of the queue, so that `add` and `offer` can be implemented by a similar rearrangement. One of the attractions of linked lists is that the insertion and removal operations implemented by these pointer rearrangements perform in constant time; by contrast, traversing a linked list is relatively slow because of the need to follow pointers from each cell to the next.

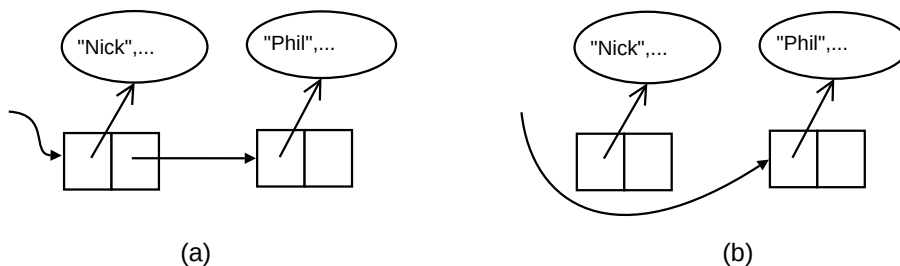


Figure 5.3: Removing an element from a `LinkedList`

You would use `LinkedList` as a `Queue` implementation in situations where thread safety isn't an issue, you don't require blocking behaviour from the queue, and where you are prepared to ensure either that your program never adds `null` elements to the queue, or else that it never uses `peek` or `poll` to determine queue emptiness (since these methods return `null` for an empty queue). The constructors for `LinkedList` are just the standard ones of Section 3.2.1.

5.2.2 AbstractQueue

Following the pattern of `Collection` and `Set`, all the other `Queue` implementations inherit from a skeletal `AbstractQueue`. The key difference between `AbstractQueue` and `LinkedList` is that `AbstractQueue` is written on the assumption that a subclass

implementation of `poll` or `peek` will return `null` only if the list is empty. In a `LinkedList`, on the other hand, `null` elements are allowed.

Code functionally equivalent to the standard implementation of `AbstractQueue` is shown in Figure 5.4. Concrete subclasses must implement the `Collection` methods `size` and `iterator`, together with the `Queue` methods `offer`, `peek` and `poll`.

```
public abstract class AbstractQueue<E>
    extends AbstractCollection<E> implements Queue<E> {

    protected AbstractQueue() { }

    public boolean add(E o) {
        if( o == null ){ throw new NullPointerException(); }
        if( ! offer(o) ){ throw new IllegalStateException("Queue full"); }
        return true;
    }

    public E remove() {
        E x = poll();
        if ( x == null ){ throw new NoSuchElementException(); }
        return x;
    }

    public E element() {
        E x = peek();
        if ( x == null ){ throw new NoSuchElementException(); }
        return x;
    }

    public void clear() { while (poll() != null) ; }
}
```

Figure 5.4: Outline implementation of `AbstractQueue`

5.2.3 PriorityQueue

`PriorityQueue` is the only other implementation besides `LinkedList` which is not designed primarily for concurrent use; it is not thread-safe nor does it provide blocking behaviour. It gives up its elements for processing according to an ordering like that used by `SortedSet`—either the natural order of its elements if they implement `Comparable`, or the ordering imposed by a `Comparator` supplied when the `PriorityQueue` is constructed. But don't be misled by this similarity into thinking that a `PriorityQueue` is like a `SortedSet` with some added operations. For one

thing, the iterator for a `PriorityQueue` is not guaranteed to return its elements in sorted order; you have to use the methods of the `Queue` interface to take advantage of the sorting. But you can use a `SortedSet` to make a `PriorityQueue`—one of the six constructors uses a `PriorityQueue` argument to supply the elements and ordering rule for the queue. The constructors are:

```

PriorityQueue()           // Natural ordering, default initial capacity (11)
PriorityQueue(Collection<? extends E> c)
                        // Natural ordering of elements taken from c
PriorityQueue(int initialCapacity)
                        // Natural ordering, specified initial capacity
PriorityQueue(int initialCapacity, Comparator<? super E> comparator)
                        // Comparator ordering, specified initial capacity
PriorityQueue(PriorityQueue<? extends E> c)
                        // Ordering and elements copied from c
PriorityQueue(SortedSet<? extends E> c)
                        // Ordering and elements copied from c

```

We can use `PriorityQueue` for a simple implementation of our to-do manager with the priority-based `Comparator` defined in section 4.2. A queue using that comparator will always yield a task chosen by the description ordering from among those which currently have the highest priority.

```

Queue<Task> priorityQueue = new PriorityQueue<Task>( priorityComparator );
priorityQueue.addAll( allTasks );
...
Task nextTask = priorityQueue.poll();

```

[Give outline of priority heap implementation of `PriorityQueue`.] `PriorityQueue` provides $O(\log n)$ time for `offer`, `poll`, `remove()` and `add`, $O(n)$ time for `remove(Object)` and `contains`, and constant time for `peek`, `element`, and `size`.

5.2.4 ConcurrentLinkedQueue

The other non-blocking `AbstractQueue` subclass is `ConcurrentLinkedQueue`, an unbounded thread-safe queue based on linked nodes. It uses FIFO ordering, and is implemented using an algorithm which allows different threads to access it for adding and removing elements simultaneously without blocking. It would be a good structure to simulate the check-in queue described above.

`ConcurrentLinkedQueue` has the two standard constructors of Section 3.2.1.

5.3 BlockingQueue

In the introduction to this chapter, we mentioned that Java 5 added a number of utility classes to the Collections Framework, for use in concurrent applications. Most of these are implementations of the `Queue` subinterface `BlockingQueue` (Figure 5.5), designed primarily to be used in producer-consumer queues.

<i>BlockingQueue<E></i>
<i>+drainTo(c : Collection<? super E>) : int</i> <i>+drainTo(c : Collection<? super E>, maxElements : int) : int</i> <i>+offer(o : E, timeout : long, unit : TimeUnit) : boolean</i> <i>+poll(timeout : long, unit : TimeUnit) : E</i> <i>+put(o : E) : void</i> <i>+remainingCapacity() : int</i> <i>+take() : E</i>

Figure 5.5: BlockingQueue

One common example of the use of producer-consumer queues is in systems which perform print spooling; client processes add print jobs to the spool queue, to be processed by one or more print service processes each of which repeatedly “consumes” the task at the head of the queue.

The key facilities which **BlockingQueue** provides to such systems are, as its name implies, enqueueing and dequeueing methods which do not return until they have executed successfully. So for example, a print server does not need to constantly poll the queue to discover whether there any print jobs waiting; it need only call the **poll** method, supplying a timeout, and the system will suspend it until either a queue element becomes available or the timeout expires. **BlockingQueue** defines seven new methods, in three groups:

Retrieving or querying the contents of the queue:

```
int drainTo(Collection<? super E> c)
    // clears the queue into c
int drainTo(Collection<? super E> c, int maxElements)
    // clears at most specified number of elements into c
int remainingCapacity()
    // returns the number of elements that would be accepted
    // without blocking
```

The effect of **drainTo** is similar to repeated calls of **poll**, with the difference that these repeated calls would each have to obtain a lock on the queue, and that between them other threads would have the opportunity to change the contents of the queue. Since **drainTo** executes atomically it will in general be much more efficient.

Adding an element:

```
boolean offer(E o, long timeout, TimeUnit unit)
    // inserts o, waiting up to the timeout
void put(E o)
    // adds o, waiting indefinitely if necessary
```

The non-blocking overload of **offer** defined in **Queue** will return **false** if it cannot immediately insert the element. This new overload waits for a time specified using

`java.util.concurrent.TimeUnit`, an `Enum` which allows timeouts to be defined in nanoseconds, microseconds, milliseconds or seconds.

Removing an element:

```
E poll(long timeout, TimeUnit unit)
    // retrieves and removes the head, waiting up to the timeout
E take()
    // retrieves and removes the head of this queue, waiting
    // indefinitely if necessary.
```

As with `offer`, the blocking overload of `poll` complements the non-blocking overload defined in `Queue`.

`BlockingQueue` implementations guarantee that their queue operations will be thread-safe and atomic. But this guarantee doesn't extend to operations inherited from `Collection`. So any thread running concurrently with others must get exclusive access to a `BlockingQueue` in order to execute any of these methods, or suffer the risk of destructive interference or—in the case of the bulk methods `addAll`, `containsAll`, `removeAll` and `retainAll`—the possibility of failing after partial completion.

5.3.1 Implementing BlockingQueue

The Collections Framework provides five implementations of `BlockingQueue`. We're going to present them here only briefly, as a detailed discussion of concurrency is outside the scope of this book.

`ArrayBlockingQueue`

This implementation is based on a *circular array*—a linear structure in which the first and last elements are logically adjacent (Figure 5.6).

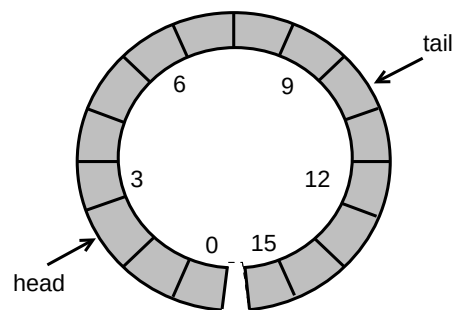


Figure 5.6: A Circular Array (only the section between head and tail-1 should be shaded)

The position labelled “head” indicates the head of the queue; each time the head element is removed from the queue, the head index is advanced. Similarly each new element is added at the tail position, resulting in that index being advanced. When either index needs to be advanced past the last element of the array, it gets the value 0. If the two indices have the same value the queue is either full or empty, so an implementation must separately keep track of the count of elements in the queue. A circular array in which the head and tail can be continuously advanced like this is better as a queue implementation than a non-circular one (*e.g.* the standard implementation of `ArrayList`, Section 6.2) in which removing the head element requires changing the position of all the remaining element so that the new head is at position 0.

Constructors for array-backed collection classes generally have a single configuration parameter, the initial length of the array. For fixed-size classes like `ArrayBlockingQueue`, this parameter is necessary in order to define the capacity of the collection. (For variable-size classes like `ArrayList`, a default initial capacity can be used, so constructors are provided that don’t require the capacity). For `ArrayBlockingQueue`, the three constructors are:

```
ArrayBlockingQueue( int capacity )
ArrayBlockingQueue( int capacity, boolean fair )
ArrayBlockingQueue( int capacity, boolean fair, Collection<? extends E> c )
```

Besides configuring the backing array, the last two also require a boolean argument to control how the queue will handle multiple blocked requests. These will occur when multiple threads attempt to remove items from an empty queue or enqueue items on to a full one. When the queue becomes able to service one of these requests, which one should it choose? The alternatives are to choose the one which has been waiting longest—that is, to implement a *fair* scheduling policy—or to choose one arbitrarily. The latter is the default because it increases efficiency of the queue operations, but it does allow the possibility that an unlucky thread might be delayed indefinitely. An `ArrayBlockingQueue` constructed with a fairness argument of `true` prevents this by processing the requests in turn.

`LinkedBlockingQueue`

If the fixed capacity of `ArrayBlockingQueue` represents a problem for your application, this class provides an alternative using the linked-list mechanism described in Section 5.2. The two standard collection constructors create a thread-safe blocking queue with a capacity of $2^{31} - 1$. A third constructor allows you to specify a lower capacity. `LinkedBlockingQueue` never implements a fair queuing strategy for blocked threads.

`PriorityBlockingQueue`

This implementation is a thread-safe, blocking version of `PriorityQueue` (Section 5.2). It uses the same priority ordering but supports multiple thread access.

PriorityBlockingQueue always handles multiple blocked threads fairly.

DelayQueue<E extends Delayed>

This is a specialised priority queue, in which the ordering is based on the *delay time* for each element—the time remaining before the element will be ready to be taken from the queue. If all elements have a positive delay time—that is, none of their associated delay times have expired—`peek` and `poll` will return null. If one or more elements has an expired delay time, it is the one with the longest-expired delay time that will be at the head of the queue. The elements of a `DelayQueue` belong to a class which implements `java.util.concurrent.Delayed`:

```
interface Delayed extends Comparable<Delayed> {
    long getDelay(TimeUnit unit);
}
```

The `getDelay` method of a `Delayed` object returns the remaining delay associated with that object. The `compareTo` method (see Section 2.2) of `Comparable` must be defined to give results consistent with the delays of the objects being compared.

For example, in our to-do manager we are likely to need reminder tasks, to ensure that we follow up e-mails and phone messages that have gone unanswered. We could define a new class `DelayedTask` as follows:

```
class DelayedTask implements Delayed {
    public final static int MILLISECONDS_IN_DAY = 60 * 60 * 24 * 1000;
    private long endTime;      // in milliseconds after January 1, 1970
    private Task task;
    DelayedTask( Task t, int daysDelay ) {
        endTime = new Date().getTime() + daysDelay * MILLISECONDS_IN_DAY;
        task = t;
    }
    public long getDelay( TimeUnit unit ) {
        long remainingTime = endTime - new Date().getTime();
        return unit.convert( remainingTime, TimeUnit.MILLISECONDS );
    }
    public int compareTo( Delayed t ) {
        long thisDelay = getDelay( TimeUnit.MILLISECONDS );
        long otherDelay = t.getDelay( TimeUnit.MILLISECONDS );
        return ( thisDelay < otherDelay ) ? -1 : ( thisDelay > otherDelay ) ? 1 : 0;
    }
    Task getTask() { return task }
}
```

and use it to implement a reminder queue:

```
BlockingQueue<DelayedTask> reminderQueue = new DelayQueue<DelayedTask>();
reminderQueue.offer( task1, 1 );
```

```

reminderQueue.offer( task2, 2 );
...
// now get the first reminder task that is ready to be processed
DelayedTask t1 = reminderQueue.poll();
if ( t1 == null ) {
    // no reminders ready yet
} else {
    // process t1
}

```

`DelayQueue` cannot implement a fair queueing strategy for blocked threads. Collection operations on a `DelayQueue` do not respect delay values, in contrast to `Queue` operations. This means, for example, that the two expressions

```

new HashSet<DelayedTask>(){ addAll( delayedTaskQueue ); }
new HashSet<DelayedTask>(){ delayedTaskQueue.drainTo( this ); }

```

will not be equal if there are any unexpired delays among the elements of `delayedTaskQueue`.

SynchronousQueue

At first sight, you might think there is little point to a queue with no internal capacity, which is a short description of `SynchronousQueue`. But in fact it can be very useful; a thread which wants to add an element to a `SynchronousQueue` must wait until another thread is ready to simultaneously take it off, and the same is true—in reverse—for a thread which wants to take an element off the queue. So the `SynchronousQueue` has the function that its name suggests, of a *rendezvous*—a mechanism for synchronizing two threads. The classic example of the use of a rendezvous is the situation in which two buffers are in use, by a writer thread which is filling one and by a reader thread which is emptying the other. When each is ready, the two buffers have to be simultaneously exchanged.

<code>//writer thread writes</code>	<code>//reader thread reads</code>
<code>//to buffer until empty</code>	<code>//from buffer until full</code>
<code>sQueue.offer(buffer); --synchronizes with--</code>	<code>Buffer b1 = buffer;</code>
<code>buffer = sQueue.take(); --synchronizes with--</code>	<code>buffer = sQueue.take();</code>
	<code>sQueue.offer(b1);</code>
<code>//writes to buffer</code>	<code>//reads from buffer</code>

As far as the `Collection` methods are concerned, a `SynchronousQueue` behaves like an empty `Collection`. `Queue` and `BlockingQueue` methods behave as you would expect for a queue with zero capacity, except for `peek`: this always returns null, even when a thread is waiting to offer an element to the queue, because a `SynchronousQueue` exists to exchange elements, not to store them.

`SynchronousQueue` has one empty constructor and one which takes a boolean parameter controlling whether the queue should implement a fair blocking strategy.

Chapter 6

Lists

Although we come to them after sets and queues, lists are probably the most-used Java collections in practice. A `List` (Figure 6.1) is a collection which—unlike a set—can contain duplicates, and which—unlike a queue—gives the user full visibility and control over the ordering of its elements.

<i>List<E></i>
<i>+add(index : int, element : E) : boolean</i> <i>+addAll(index : int, c : Collection<? extends E>) : boolean</i> <i>+indexOf(o : Object) : int</i> <i>+lastIndexOf(o : Object) : int</i> <i>+listIterator(index : int) : ListIterator<E></i> <i>+listIterator() : ListIterator<E></i> <i>+set(index : int, element : E) : E</i> <i>+remove(index : int) : E</i> <i>+get(index : int) : E</i> <i>+sublist(fromIndex : int, toIndex : int) : List<E></i> ...

Figure 6.1: List

In addition to the operations inherited from `Collection`, the `List` interface includes operations for:

Positional Access: manipulate elements based on their numerical position in the list:

```
void add( int index, E element )           // add element o at given index
boolean addAll( int index, Collection<? extends E> c )
                                           // add contents of c at given index
E get( int index )                         // return element with given index
```

```

E remove( int index )           // remove element with given index
E set( int index, E element )   // replace element with given index

```

Search: search for a specified object in the list and return its numerical position. These methods return -1 if the object is not present.

```

int indexOf(Object o)           // return index of first occurrence of o
int lastIndexOf(Object o)       // return index of last occurrence of o

```

Range-view: get a view of a range of the list.

```

List<E> subList(int fromIndex, int toIndex)
// return a view of a portion of the list

```

`subList` works like the subsequence operations on `SortedSet` (Section 4.2): the returned list contains the list elements starting at `fromIndex` up to but not including `toIndex`. The returned list has no separate existence—it is just a view of part of the the list from which it was obtained, so changes in it are reflected in the original list, and vice versa.

List iteration: extend `Iterator` semantics to take advantage of the list's sequential nature.

```

ListIterator<E> listIterator() // returns a ListIterator for this list
ListIterator<E> listIterator( int index) // returns a ListIterator for this list,
// starting at the given index

```

`ListIterator` is an interface which adds methods to `Iterator` to traverse a list backwards, to change list elements or add new ones, and to get the the current position of the iterator. The current position of a `ListIterator` always lies between two elements, the one that would be returned by a call to `previous` and the one that would be returned by a call to `next`. In a list of length n , there are $n + 1$ valid index values, from 0 (before the first element) to n (after the last one). To the `Iterator` methods `hasNext`, `next` and `remove`, `ListIterator` adds the following methods:

```

public interface ListIterator<E> extends Iterator<E> {
    void add(E o);           // Inserts the specified element into the list
    boolean hasPrevious();   // Returns true if this list iterator has further
                             // elements in the reverse direction.
    int nextIndex();         // Returns the index of the element that would be
                             // returned by a subsequent call to next.
    E previous();           // Returns the previous element in the list.
    int previousIndex();     // Returns the index of the element that would be
                             // returned by a subsequent call to next
    void set(E o);          // Replaces the last element returned by next or
                             // previous with the specified element
}

```

6.1 Using the Methods of List

Let's look at examples of the use of these methods. You may remember that although queues—especially `PriorityQueue`—looked like a plausible implementations for our to-do manager, they have one severe drawback: there is no way in general to inspect or manipulate any element other than the one at the head. We found one example of this drawback in the situation in which we were looking for a short task to fit into a tight time-slot, but there are many others. Lists provide low-level facilities for manipulating their contents, but at a price: we have to do more low-level work.

For example, there is no list implementation corresponding to `PriorityQueue` which will organise our tasks by priority. We will have to do it for ourselves, in this case by creating a separate task list for each priority. For the moment, we will keep the references to these separate lists in another list which we can initialise like this:

```
List<List<Task>> taskLists = new ArrayList<List<Task>>();
for( Priority p : Priority.values() ) {
    taskLists.add( new ArrayList<Task>() );
}
```

and to add a task `sampleTask` we can write

```
Priority p = sampleTask.getPriority();
taskLists.get(p.ordinal()).add(sampleTask);
```

Now we can, for example, carry out all of the phone tasks in turn, in descending order of priority.

```
for( List<Task> taskList : taskLists ) {
    for( Task t : taskList ) {
        if( t instanceof PhoneTask ) {
            // process t
        }
    }
}
```

For the ordering of tasks within each of the fixed-priority lists, we will treat the list like a queue having the head element at index 0 and the tail element at index $n - 1$, where n is the number of outstanding tasks. `offer` can then be simulated by `add(E o)`—which appends the supplied element to the end of the list—and the methods which retrieve the head of the queue can be simulated by `get(0)`. With this understanding, let's look at examples of the use of methods from each of the groups above.

Positional Access: although in the normal way of things new tasks will be added on to the end of each of the task lists, there may be tasks that just have to go to the head of the queue:

```
List<Task> highPriorityTaskList = taskLists.get( Priority.HIGH.getOrdinal() );
highPriorityTaskList.add( 0, new PhoneTask( "home", Priority.HIGH, "321 9876" ) );
```

or if—the next example is sadly all too frequent!—we have to do all the tasks left over from Monday before we can start on Tuesday’s:

```
tuesdayHighPriorityTasks.addAll( 0, mondayHighPriorityTasks );
```

We often like to inspect the next task, without necessarily committing to doing it immediately (as with the `peek` method of `Queue`):

```
Task nextTask = highPriorityTaskList.get( 0 );
```

but if we are really confident that we can do it, we can simultaneously retrieve it and remove it from the list (as with the `poll` method of `Queue`). Note that `remove` adjusts the index of all subsequent elements, so the `List` will still begin at index 0.

```
Task nextTask = highPriorityTaskList.remove( 0 );
```

But the only real answer to our problems is not to try to do so many things. As a start we’re just going to drop the last activity on the low-priority list—the last one we aim to do in the day. Better still, let’s reward ourselves for actually getting to that point by scheduling a more agreeable replacement “task”:

```
Task dayEndTask = new Task( "go to pub", Priority.LOW );
lowPriorityTaskList.set( lowPriorityTaskList.size() - 1, moreImportantTask );
```

The `set` method actually returns a reference to the object being replaced, but to really follow through on our decision that “less is more”, we have to steel ourselves to throw that reference away on this occasion.

Search: We often absent-mindedly insert the same task into the system more than once, so it would be useful to be able to remove the duplicate entries. One way would be to identify the first occurrence of a task and then repeatedly remove the last occurrence as identified by `lastIndexOf`.

```
for( int i = 0 ; i < taskList.size() ; i++ ) {
    // get the next task to de-duplicate
    Task task = taskList.get(i);
    while( i != taskList.lastIndexOf( task ) ) {
        taskList.remove( taskList.lastIndexOf( task ) );
    }
}
```

Your first thought for this example might have been to use *foreach* or (equivalently) an explicit iterator to work through the tasks to be de-duplicated. But recall from Section 2.6.1 that if the iterator on `taskList` is fail-fast, that code would fail with `ConcurrentModificationException`. This is an example of a sound program caught up in the blanket restriction on “concurrent” modification.

Range-view: If we wanted to build co-operative working into the new task manager, we might define a method to allow us to share task lists:

```
public void shareTasks( List<List<Task>> taskLists )
```

but we might not always want to share *all* of the task lists. Suppose, for example that we wanted to supply a list to `shareTasks` consisting of only the top two priority lists. We would need to create a view of our `List<List<Task>>` containing only those two elements, like this:

```
shareTasks( taskList.subList( 0, 2 ) )
```

List iteration: The way that we removed duplicate tasks using `lastIndexOf` was simple but inefficient. We repeatedly found the last occurrence of an element by searching for it from the very end of the list. A list iterator will allow us to do the job more efficiently, resuming the search each time from the point at which the last duplicate element was found.

```
for( int i = 0 ; i < taskList.size() ; i++ ) {
    Task task = taskList.get(i);
    // get a ListIterator, initially pointing past the end of the list
    ListIterator<Task> iter = taskList.listIterator( taskList.size() );
    Task t1 = iter.previous();
    // now search backwards removing duplicates, and stopping when we
    // reach the original task
    while( task != t1 ) {
        if ( task.equals( t1 ) ) {
            iter.remove();
        }
        t1 = iter.previous();
    }
}
```

6.2 Implementing List

There are three concrete implementations of `List` in the Collections Framework, differing again, as with the implementations of `Set`, in how fast they perform the various operations defined by the interface; in this case, however, there are no functional differences between the implementations. Figure 6.2 shows how they are related. In this section we look at each implementation in turn, and in Section 6.3 we provide a performance comparison.

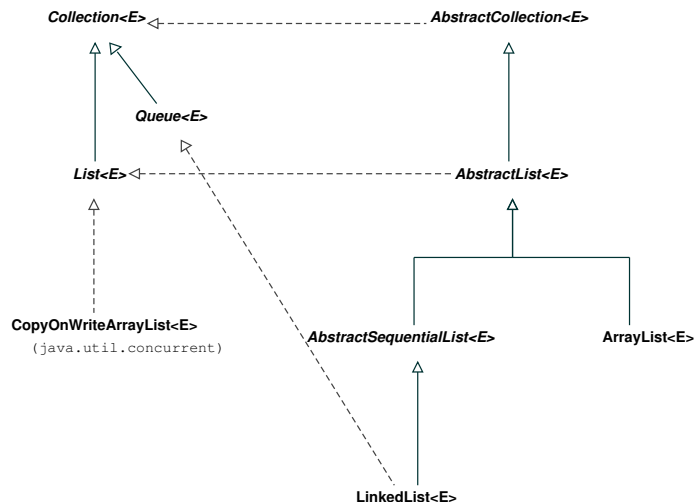


Figure 6.2: Implementations of the `List` interface (extracted from the front endpaper)

6.2.1 AbstractList

The Collections Framework again provides a skeletal abstract implementation, **AbstractList**, as a basis for concrete implementations. Figure 6.4 shows an outline implementation of **AbstractList**, although for reasons of space it omits the implementation of `subList`.

Most of the methods on the **List** interface are implemented in **AbstractList**, although `set`, `remove` and `add` simply throw **UnsupportedOperationException**, to be overridden in subclasses which implement modifiable **Lists**. The other methods are implemented in terms of `get`, which is left abstract and as a non-optional operation must be implemented by any subclass. Of the methods inherited from **AbstractCollection**, `size` is still abstract but `iterator` returns an instance of an inner class whose `next` method relies on `get`.

So for direct subclasses of **AbstractList** the efficiency of iteration depends on the efficiency of random access. That's appropriate for the first class we will look at, which uses arrays—of all linear data structures the fastest for random access—to provide an implementation of **List**. It's exactly the wrong thing to do for classes dependent on sequential access, though, so **AbstractSequentialList** switches the dependency around. List classes themselves will always know whether they are a sequential or random implementation, of course, but generic algorithms (Chapter 9), which often have a sequential and a random variant, need a hint in order to be able to adopt the right one. That hint is provided by the **RandomAccess**, a marker

interface (with no methods) which is implemented by all the `List` implementations which are backed by arrays.

A very simple example of the use of `AbstractList` is given by the class `CopiesList` (Figure 6.3) which is used within the `Collections` class to implement the method `nCopies` (Section 8.1). `CopiesList` behaves as an immutable `List` view which appears to contain a fixed number of copies of a given element, implementing all the non-optional methods of the `List` interface although it physically contains only a single element.

```
class CopiesList<E> extends AbstractList<E> {
    private int n;
    private E element;
    CopiesList(int n, E o) {
        if (n < 0) {
            throw new IllegalArgumentException("List length = " + n);
        }
        this.n = n;
        element = o;
    }
    public int size() { return n; }
    public boolean contains(Object obj) {
        return n != 0 && eq(obj, element);
    }
    public E get(int index) {
        if (index < 0 || index >= n) {
            throw new IndexOutOfBoundsException("Index: "+index+", Size: "+n);
        }
        return element;
    }
}
```

Figure 6.3: Outline implementation of `CopiesList`

6.2.2 ArrayList

Arrays are provided as part of the Java language and have a very convenient syntax, but their key disadvantage—that once created they cannot be resized—makes them increasingly less popular than `List` implementations which (if modifiable at all) are indefinitely extensible. The most commonly-used implementation of `List` is in fact `ArrayList`—that is, a `List` backed by an array.

The standard implementation of `ArrayList` stores the `List` elements in contiguous array locations, with the first element always stored at index 0 in the array. It requires an array at least large enough (with sufficient *capacity*) to contain the ele-

```

public abstract class AbstractList<E> extends AbstractCollection<E> implements List<E> {

    // Object method overrides
    public boolean equals(Object o) {
        if (!(o instanceof List)) return false;
        List l = (List) o;
        if ( size() != l.size() ) return false;
        for ( int i = 0 ; i < size() ; i++ ) {
            if ( get(i) == null ? l.get(i) != null : ! get(i).equals( l.get(i) ) ) { return false; }
        }
        return true;
    }
    public int hashCode() {
        int h = 1;
        for( E el : this ) h += h * 31 + ( el==null ? 0 : el.hashCode() );
        return hashCode;
    }

    // Collection method implementations
    public boolean add(E o) {
        add(size(), o);
        return true;
    }
    public Iterator<E> iterator() { return new Itr(); }

    // List method implementations
    public E set(int index, E element) { throw new UnsupportedOperationException(); }
    public E remove(int index) { throw new UnsupportedOperationException(); }
    public void add(int index, E element) { throw new UnsupportedOperationException(); }
    public boolean addAll(int index, Collection<? extends E> c) {
        for( E el : c ) { add(index++, el); }
        return c.size() > 0;
    }
    public abstract E get( int index );
    public int indexOf(Object o) {
        ListIterator<E> e = listIterator();
        while( e.hasNext() ) {
            E el = e.next();
            if ( o == null ? el == null : o.equals( el ) ) { return e.previousIndex(); }
        }
        return -1;
    }
    public int lastIndexOf(Object o) {
        ListIterator<E> e = listIterator(size());
        while( e.hasPrevious() ) {
            E el = e.previous();
            if ( o == null ? el == null : o.equals( el ) ) { return e.nextIndex(); }
        }
        return -1;
    }
    public ListIterator<E> listIterator() { return listIterator(0); }
    public ListIterator<E> listIterator(final int index) {
        if (index<0 || index>size()) throw new IndexOutOfBoundsException();
        return new ListItr(index);
    }
    public List<E> subList(int fromIndex, int toIndex) { /* ... */ }
}

```

```

// Iterator classes
private class Itr implements Iterator<E> {
    int cursor = 0;
    int lastRet = -1;
    public boolean hasNext() { return cursor != size(); }
    public E next() {
        try {
            E next = get(cursor);
            lastRet = cursor++;
            return next;
        } catch (IndexOutOfBoundsException e) { throw new NoSuchElementException(); }
    }
    public void remove() {
        if (lastRet == -1) throw new IllegalStateException();
        AbstractList.this.remove(lastRet);
        if (lastRet < cursor) cursor--;
        lastRet = -1;
    }
}

private class ListItr extends Itr implements ListIterator<E> {
    ListItr(int index) { cursor = index; }
    public boolean hasPrevious() { return cursor != 0; }
    public E previous() {
        try {
            int i = cursor - 1;
            E previous = get(i);
            lastRet = cursor = i;
            return previous;
        } catch (IndexOutOfBoundsException e) { throw new NoSuchElementException(); }
    }
    public int nextIndex() { return cursor; }
    public int previousIndex() { return cursor-1; }
    public void set(E o) {
        if (lastRet == -1) throw new IllegalStateException();
        AbstractList.this.set(lastRet, o);
    }
    public void add(E o) {
        AbstractList.this.add(cursor++, o);
        lastRet = -1;
    }
}
}

```

Figure 6.4: Outline implementation of `AbstractList`

ments, together with a way of keeping track of the number of “occupied” locations (the size of the `List`). If a list has grown to the point where its size equals to its capacity, attempting to add another element will require the `ArrayList` to replace the backing array with a larger one capable of holding the old contents and the new element, and with a margin for further expansion (the standard implementation of `ArrayList` actually uses a new array double the length of the old one).

The performance of `ArrayList` reflects array performance for “random-access” operations: `set` and `get` take constant time. And `ArrayList` inherits `AbstractList`’s iterator, whose `next` method is defined in terms of `get`, so that too takes constant time. The downside of an array implementation is in inserting or removing elements at arbitrary positions, because that may require adjusting the position of other elements. For example, Figure 6.5(a) shows a new `ArrayList` after three elements have been added by means of the following statement:

```
new ArrayList<Character>(){ add('a'); add('b'); add('c'); };
```

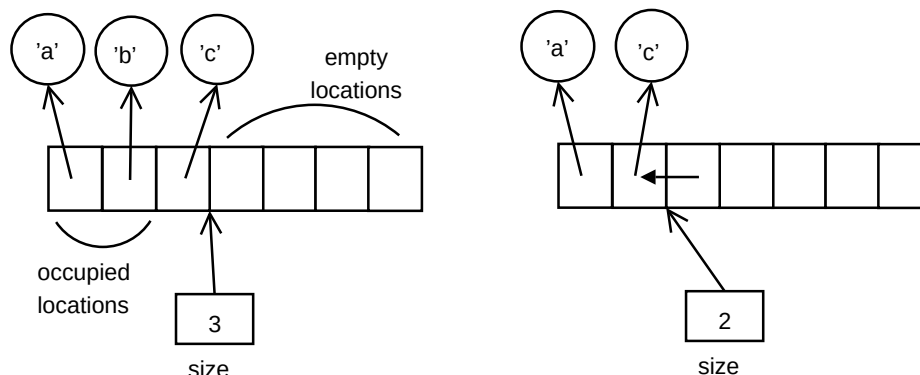


Figure 6.5: Removing an element from an `ArrayList`

If we now want to remove the element at index 1 of an array, the implementation must preserve the order of the remaining elements and also ensure that the occupied region of the array is still to start at index 0. So the element at index 2 must be moved to index 1, that at index 3 to index 2, and so on. Figure 6.5(b) shows our sample `ArrayList` after this operation has been carried out. Since every element must be moved in turn, the time complexity of this operation is proportional to the size of the list (although it is worth noting that because this operation can usually be implemented in hardware, the constant factor is low).

Even so the alert reader, recalling the discussion of the circular array used to implement `ArrayBlockingQueue` (Section 5.2), may wonder why a circular array was not chosen for the implementation of `ArrayList` too. It is true that the `add` and `remove` methods of a circular array only show a much better performance when they are called with an index argument of 0, but this is such a common case and the overhead of using a circular array is so small, that the question remains.

As we mentioned in the discussion of `ArrayBlockingQueue` (Section 5.2), variable-size array-backed collection classes can have one configuration parameter—the initial length of the array. So besides the standard Collection Framework constructors, `ArrayList` has one which allows you to choose the value of the initial capacity to be large enough to accommodate the elements of the collection without frequent create-copy operations. If you choose one of the others, the standard implementation uses an initial capacity of 10.

6.2.3 AbstractSequentialList

In discussing `AbstractList` we mentioned that the efficiency of iteration depends on the efficiency of the `get` method. This is appropriate for a `List` implementation like `ArrayList`, which is backed by a data structure that implements `get` efficiently on random index values. A class like `LinkedList`, on the other hand (which we met as a `Queue` implementation in Section 5.2) can only access its data sequentially, so implementing iteration in terms of `get` is exactly the wrong way round. `AbstractSequentialList` is designed for situations like this; it overrides the `AbstractList` implementation of `listIterator` with an abstract declaration, and replaces the abstract `get` method of `AbstractList` with a concrete definition in terms of iteration over the list.

To implement a sequential list using this class you need to override the `listIterator` and `size` methods and, if the list is to be modifiable, implement the `add`, `set` and `remove` methods of the iterator.

6.2.4 LinkedList

We learnt most of what we need to know about `LinkedList` when we considered it as a `Queue` implementation (Section 5.2). The only feature that was not relevant to discuss in the context of queues is the double linking (Figure 6.6) that makes `hasPrevious` and `previous` as efficient as `hasNext` and `next`. `LinkedList` can add and remove elements in constant time, but random access must be implemented through iteration.

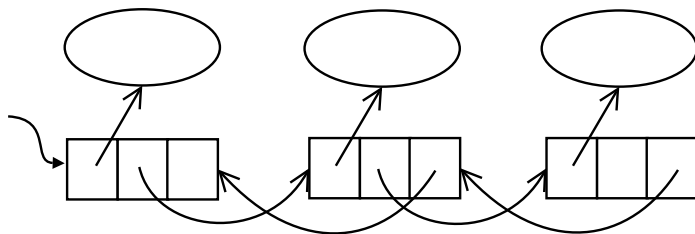


Figure 6.6: A Doubly-Linked List

6.2.5 CopyOnWriteArrayList

In Section 4.1 we met `CopyOnWriteArraySet`, a set implementation designed to provide thread-safety together with very fast read access. `CopyOnWriteArrayList` is a `List` implementation with the same design aims. This combination of thread safety with fast read access is useful in some concurrent programs, especially when a collection of observer objects needs to receive frequent event notifications. The cost is that the collection has to be immutable, and that therefore a new one must be created if any changes are made. This cost may not be too high to pay if changes in the set of observers occur only rarely.

`CopyOnWriteArraySet` in fact delegates all its operations to an instance of `CopyOnWriteArrayList`. There are two differences between these two classes; one is that the `add` and `addAll` methods of `CopyOnWriteArraySet` are implemented in terms of methods `addIfAbsent` and `addAllAbsent` provided by `CopyOnWriteArrayList` to ensure that the set contains no duplicates. The other difference is that `CopyOnWriteArrayList` has an extra constructor—in addition to the two standard ones (page 30)—that allows it to be created using the elements of a supplied array as its initial contents.

6.3 Comparison of List Implementations

As always, the O -formulae in Figure 6.7 don't tell the whole story, not least because of the constant factors they omit. For example, although element addition and removal in an `ArrayList` take linear time, the element copying operation can usually be implemented in hardware and is therefore very fast. By contrast, although calling `next` in a `LinkedList` takes constant time, a pointer has to be followed so the constant is quite large.

	get	add	contains	next	notes
<code>ArrayList</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$	
<code>LinkedList</code>	$O(n)$	$O(1)$	$O(n)$	$O(1)$	
<code>CopyOnWriteArrayList</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$	

Figure 6.7: Comparative performance of different `List` implementations

The cost of adding an element to an `ArrayList` includes an allowance for the cost of expanding the list when the array is full. This is an example of an amortized cost (Section 2.4): the cost of transferring the contents of the old array to the new one is linear in the number of elements in the old array. This cost has to be incurred repeatedly for each element as the array continues to grow in size, but if each resize doubles the size of the array (or multiplies it by any constant factor), the frequency of resizing continually decreases. Overall, the total cost of resizing the `ArrayList` for N elements is $O(N)$, so the amortized cost is $O(1)$.

Chapter 7

Maps

`Map` is the last of the major Collection Framework interfaces in this chapter, and the only one which does not inherit from `Collection`. It defines the operations which are supported by a set of key-to-value associations in which the keys are unique. These operations are shown in Figure 7.1 and fall into five groups, broadly parallel to three of the operation groups of `Collection`—adding elements, removing elements, and querying collection contents. The fourth operation group of `Collection` contains `iterator` and `toArray`, for which `Map` has no corresponding methods.

<i>Map<K,V></i>
<i>+clear()</i> <i>+containsKey(key : Object) : boolean</i> <i>+containsValue(value : Object) : boolean</i> <i>+entrySet() : Set<Map.Entry<K,V>></i> <i>+get(key : Object) : boolean</i> <i>+isEmpty() : boolean</i> <i>+keySet() : Set<K></i> <i>+put(key : K, value : V) : V</i> <i>+putAll(t : Map<? extends K, ? extends V>)</i> <i>+remove(key : Object) : V</i> <i>+size() : int</i> <i>+values() : Collection<V></i>

Figure 7.1: The `Map` interface (extracted from the back endpaper)

Adding associations:

```
V put( K key, V value )           // adds or replaces a key-value association.
```

```

// Returns the old value (may be null) if the
// key was present, otherwise returns null
void putAll(Map<? extends K,           // adds each of the key-value associations in
            ? extends V> t)          // the supplied map into the receiver

```

Removing associations:

```

void clear()                // removes all associations from this map
V remove(Object key)        // removes the association, if any, with the given
                             // key; returns the value with which it was associated,
                             // or null

```

You may recall from Section 3.1 that the `remove` method of `Collection` also has a parameter of type `Object`. Since the contract for `remove` provides for it to do nothing if the key is not present in the map, there can be no harm in allowing a non-generic parameter here.

The operations in this and the previous group, to add and remove associations, are optional; calling them on an unmodifiable map will not change the map and may result in an `UnsupportedOperationException`.

Querying for the existence of presence of specific keys, values, or associations:

```

boolean containsKey(Object k) // returns true if k is present as a key. k may be null
                             // for maps that allow null keys.
boolean containsValue(Object v) // returns true if v is present as a value. v may be null.
V get(Object k)                // returns the value corresponding to k, or null if k is
                             // not present as a key.

```

Retrieving the keys, values or associations in bulk:

```

Set<Map.Entry<K, V>> entrySet() // returns a set view of the key-value associations.
Set<K> keySet()                 // returns a set view of the keys
Collection<V> values()          // returns a set view of the values

```

The collections returned by these methods are backed by the map, so any changes to them are reflected in the map itself. In fact, only limited changes can be made: elements can be removed from them, either directly or via an iterator, but cannot be added. Removing a key removes the single corresponding key-value association; removing a value, on the other hand, removes only one of the associations mapping to it; the value may still be present as part of an association with a different key.

Querying the association count

```

int size()                   // returns the number of key-value associations
boolean isEmpty()            // returns true if there are no key-value associations

```

As with the `size` method of `Collection`, the largest element count that can be reported is `Integer.MAX_VALUE`.

7.1 Using the Methods of Map

In the last section we modelled the outstanding tasks with a `List<List<Task>>`, placing the list of lowest-priority tasks at index 0, the next-lowest at position 1, and so on. This feels clumsy, especially when we see that to retrieve a task list we have to write

```
List<Task> highPriorityTaskList = taskLists.get( Priority.HIGH.getOrdinal() );
```

Now that we have `Maps`, we can improve on using a list of lists by instead mapping priorities to tasklists:

```
Map<Priority,List<Task>> taskMap = new HashMap<Priority,List<Task>>();
for( Priority p : Priority.values() ) {
    taskMap.put( p, new ArrayList<Task>() );
}
```

And to get the highest-priority task list as above we can now write, rather more neatly,

```
List<Task> highPriorityTaskList = taskMap.get( Priority.HIGH );
```

To see the use of some of the other methods of `Map`, let's extend the example a little to allow for the possibility that some of these tasks might actually earn us some money by being billable. One way of representing this would be by defining a class `Client`:

```
class Client {...}
Client c1 = new Client(...);
```

and creating a mapping from task details to client objects:

```
Map<String,Client> billingMap = new HashMap<String,Client>();
billingMap.put( "code ui", c1 );
```

How can we ensure that the system can still handle non-billable tasks? There is a choice: we can either simply not add the name of a non-billable task into the `billingMap` or, alternatively, we can map it to `null`. Let's take the first alternative, so that as part of the code for processing a task `t` we can write

```
String taskDetails = t.toString();
if ( billingMap.containsKey( taskDetails ) ) {
    Client client = billingMap.get( taskDetails );
    client.bill(...);
}
```

When finally we have finished all the work we were contracted to do by our client Acme Corp., the mappings which associate task details with Acme can be removed:

```

Client acme = ...
Collection<Client> clients = billingMap.values();
for( Iterator<Client> iter = clients.iterator() ; iter.hasNext() ; ) {
    if (iter.next().equals( acme )) {
        iter.remove();
    }
}

```

7.2 Implementing Map

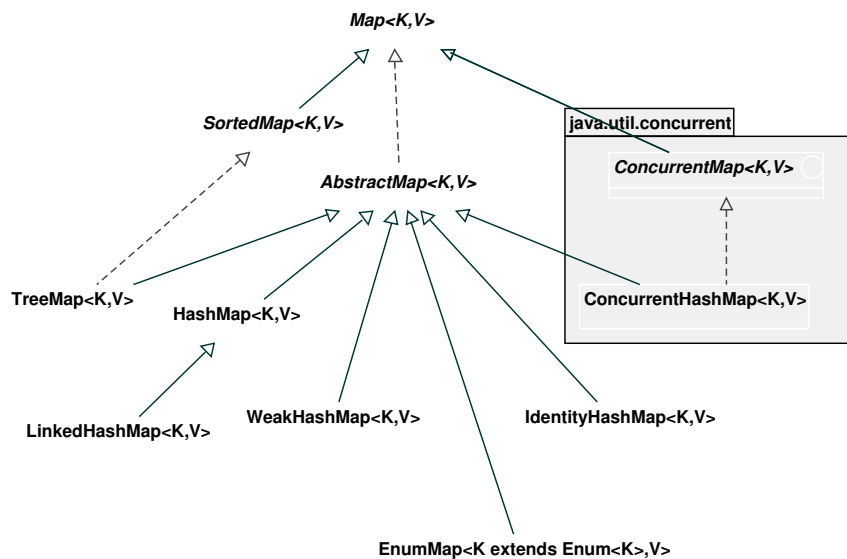


Figure 7.2: The structure of `Map` implementations in the Collections Framework (extracted from the front endpaper)

The implementations that the Collections Framework provides for `Map` are shown in Figure 7.2. As with the other main interfaces, the Framework provides a skeletal implementation (`AbstractMap`) for subclassing, together with seven concrete implementations. We shall discuss `HashMap`, `LinkedHashMap`, `WeakHashMap` and `IdentityHashMap` here; `EnumMap` is specialised for use with enums and is discussed with them in Section ??, and `SortedMap` and `ConcurrentMap` are discussed along with their implementations in the sections following this one.

For constructors, the general rule for `Map` implementations is parallel to that for `Collection` subclasses. Every general-purpose implementation (excluding `EnumMap`, that is) has at least two constructors; taking `HashMap` as an example they are

```
public HashMap()
public HashMap( Map<? extends K,? extends V> m )
```

The first of these creates an empty map, and the second a map that will contain the key-value mappings contained in the supplied map `m`. The keys and values of `m` must have types which are the same as of the keys and values, respectively, of the map being created—or subtypes of them, of course. Using this second constructor has the same effect as creating an empty map with the default constructor, then adding the contents of `m` using `putAll`. In addition to these two, the standard implementations have other constructors for configuration purposes.

Iterators returned by most of the standard `Map` implementations are fail-fast in the same way as `List` iterators are (Section 6.1). The exceptions are `ConcurrentHashMap` and `EnumMap`, whose iterators avoid throwing `ConcurrentModificationException`; as a result, the view of the map that they reflect may not include modifications that have occurred since they were created.

7.2.1 AbstractMap

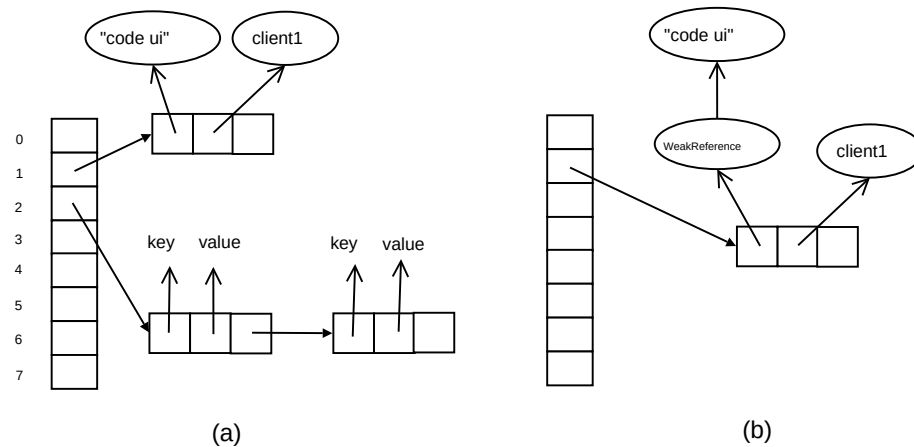
[tbs: outline implementation of `AbstractMap`]. Most of the methods on the `Map` interface are implemented in `AbstractMap`, although `clear`, `put`, `putAll`, and `remove` simply throw `UnsupportedOperationException`, to be overridden in subclasses which implement modifiable `Maps`. The other methods are implemented in terms of `entrySet`, which is left abstract and as a non-optional operation must be implemented by any subclass.

Notice how much this limits the practical usefulness of `AbstractMap`; if all access to its data has to be through a set of key-value mappings, then operations like `get` can only be implemented by iterating through that set to find the right key. We know from the discussion of `HashSet` (Section 4.1) that hash tables can give much better performance than that, so it is not surprising to find that almost all the standard implementations of `Map` are hash-based and therefore override virtually all the methods of `AbstractMap`.

7.2.2 HashMap

In discussing set implementation in Section 4.1 we mentioned that `HashSet` delegates all its operations to a private instance of `HashMap`. Figure 7.3(a) repeats Figure 4.4 without the simplification that removed the value elements from the map (all elements in a `HashSet` are stored as keys with the same constant value). The discussion in Section 4.1 of hash tables and their performance applies equally to `HashMap`. In particular, without key collisions it provides constant-time performance for `put` and `get`. Iteration over a collection of keys or values requires time proportional to its capacity plus the number of key-value mappings that it contains.

The extra constructors for `HashMap` are also like those of `HashSet`, allowing the programmer to specify the initial capacity and, optionally, the load factor at which the table will be rehashed.

Figure 7.3: `HashMap` and `WeakHashMap`

7.2.3 `LinkedHashMap`

Like `LinkedHashSet`, `LinkedHashMap` refines the contract of its parent class `HashMap`, by guaranteeing that its iterators will return their elements in the order in which they were first added. In the same way, iteration over a `LinkedHashMap` takes time proportional only to the number of elements in the map, not its capacity. Its constructors are like those of `HashMap`.

7.2.4 `WeakHashMap`

An ordinary `Map` keeps ordinary (“strong”) references to all the objects it contains. That means that even when a key has become unreachable it cannot be garbage collected. Normally, that’s exactly what we want; in the previous section, where we mapped task details to client objects, we wouldn’t want a mapping to disappear just because we weren’t holding a reference to the string object that we had put into the `HashMap` to represent the task details. To look up the value associated with a supplied key, the `HashMap` will look for a key which matches (in the sense of `equals`) the supplied one—they don’t have to be physically the same object.

But suppose that the objects of the key class are unique—that is, no two objects of that class ever are equal in the sense of `equals`. For example, each object might contain a unique serial number. In this case, once we no longer have a reference—from outside the map—to an object being used as a key, we can never look it up again, because we can never recreate it. So the map might as well get rid of the key-value pair and, in fact, there may be a strong advantage in doing so if the map is large and memory is in short supply. That is the purpose that `WeakHashMap` accomplishes.

Internally `WeakHashMap` holds references to its key objects through references of the class `java.lang.ref.WeakReference` (Figure 7.3(b)). A `WeakReference` introduces an extra level of indirection in reaching an object. For example, to make a weak reference to an object `myObj` you would write

```
WeakReference wref = new WeakReference( myObj );
```

and at a later time recover a strong reference to it using the `get` method of `WeakReference`:

```
Object myObjRecoveredRef = wref.get();
```

If the reference variable `myObj` is nulled or goes out of scope and there are no other strong references to the object it referred to, then the existence of the weak reference will not by itself prevent the garbage collector from reclaiming the object. So the recovered reference value `myObjRecoveredRef` may (or may not) be null.

`WeakHashMap` uses `WeakReferences` to refer to its keys. Before most operations on a `WeakHashMap` are executed, the map checks which keys have been reclaimed (the `WeakReference` mechanism allows you to tell the garbage collector to leave you information each time it reclaims a weakly referenced object). For each key which the garbage collector has reclaimed, the `WeakHashMap` removes the entire key-value association.

What is a `WeakHashMap` good for? Imagine you were using a map to associate extra information with objects and you had a high turnover of objects; you would need to remember to remove each key object to prevent the map growing indefinitely and causing a memory leak. If the map were a `WeakHashMap`, though, neglecting to remove unwanted keys wouldn't be a serious problem, as they would eventually be removed by the garbage collector. So one use is to guard against memory leaks.

A more general application is in those situations where you don't mind information disappearing if memory is low, like a cache. Here `WeakHashMap` is useful whether or not the keys are unique, because you can always recreate a key if necessary to see if the corresponding value is still in the cache. `WeakHashMap` isn't perfect for this purpose—because it weakly references the map's keys rather than its values (usually much larger), it gives the garbage collector the wrong information about the gains that it can make. A second drawback is that weak references are *too* weak; the garbage collector is liable to reclaim a weakly reachable object at any time, and the programmer cannot influence this in any way. (A sister class of `WeakReference`, `java.lang.ref.SoftReference`, is treated differently: the garbage collector should postpone reclaiming these until it is under severe memory pressure. Dr. Heinz Kabutz has written a `SoftReference`-based map using generics; see <http://www.javaspecialists.co.za/archive/Issue098.html>).

`WeakHashMap` performs similarly to `HashMap` though more slowly because of the overheads of the extra level of indirection for keys. The cost of clearing out unwanted key-value associations before each operation is proportional to the number of associations that need to be removed.

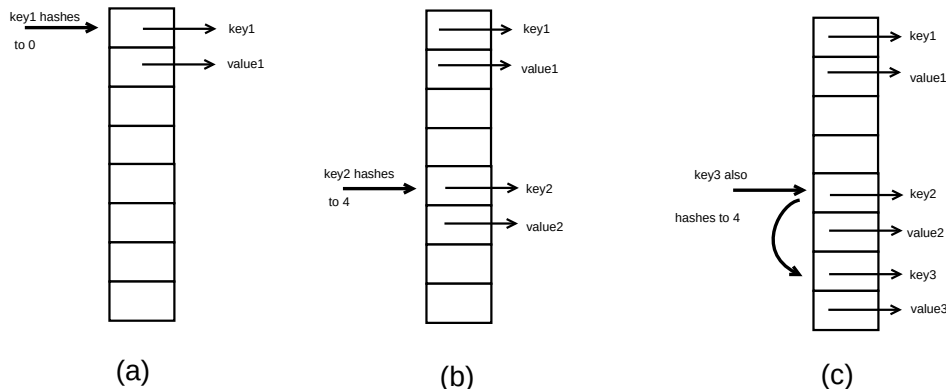


Figure 7.4: Resolving Collisions by Linear Probing

7.2.5 IdentityHashMap

An `IdentityHashMap` differs from an ordinary `HashMap` in that two keys are considered equal only if they are physically the same object: identity, rather than `equals`, is used for key comparison (in contravention of the contract for `Map`). This is a specialised class, commonly used in operations like serialization, in which a graph has to be traversed and information stored about each node. The algorithm used for traversing the graph must be able to check, for each node it encounters, whether that node has already been seen; otherwise graph cycles could be followed indefinitely. An ordinary map can only detect duplication by equality, and of course a graph could contain two different but equal nodes. An `IdentityHashMap`, by contrast, will report a node as being present only if that same node has previously been put into the map.

The standard implementation of `IdentityHashMap` handles collision differently from the chaining method shown in Figure 4.4 and used by all the other variants of `HashSet` and `HashMap`. This implementation uses a technique called *linear probing*, in which the key and value references are stored directly in adjacent locations in the table itself, rather than in cells referenced from it. In the standard implementation, collisions are handled by simply looking stepping along the table until the first free pair of locations is found. Figure 7.4 shows three stages in filling an `IdentityHashMap` with a capacity of 8. In (a) we are storing a key-value pair whose key hashes to 0, and in (b) a pair whose key hashes to 4. The third key, added in (c), also hashes to 4, so the algorithm steps along the table until it finds an unused

location. In this case the first one it tries, with index 6, is free and can be used. Deletions are more tricky than with chaining; if key2 and value2 were removed from the table in Figure 4.4 key3 and value3 would have to be moved down to take their place.

There are three constructors for `IdentityHashMap`:

```
public IdentityHashMap()
public IdentityHashMap( Map<? extends K,? extends V> m )
public IdentityHashMap(int expectedMaxSize )
```

The first two are the standard constructors that every general-purpose `Map` implementation has. The third takes the place of the two constructors which in other `HashMaps` allow the user to control the initial capacity of the table and the load factor at which it will be rehashed. `IdentityHashMap` doesn't allow this, fixing it instead (at .67 in the standard implementation) in order to protect the user from herself: whereas the cost of finding a key in a table using chaining is proportional to the load factor l , in a table using linear probing it is proportional to $1/(1-l)$. So avoiding high load factors is too important to be left to the programmer!

Out of all the Collection Framework hash implementations why does `IdentityHashMap` alone use linear probing when all the others use chaining? Again, it is to protect the programmer from herself. Linear probing has a better performance than chaining under some circumstances, but its performance deteriorates dramatically if the hashing algorithm being used doesn't distribute keys evenly over the table. `IdentityHashMap` is the only implementation for which the user can't supply the hashing algorithm—it uses `System.identityHashCode`, which gives the same hash code as would be returned by the `Object` method `hashCode`, whether or not that method is overridden in the object being hashed.

7.3 SortedMap

Like `SortedSet`, the subinterface `SortedMap` (see back endpaper) adds to its contract a guarantee that its iterators will traverse the map in ascending key order. Its definition is closely similar to that of `SortedSet`, with methods like `firstKey` and `headMap` corresponding to the `SortedSet` methods `first` and `headSet`. Again you can either use the natural ordering of the keys or impose a `Comparator` on them, but in any case the `compare` method must be consistent with `equals`, as the `SortedMap` will use it in its place.

The extra methods defined by the `SortedMap` interface fall into three groups:

Retrieving the comparator:

```
Comparator<? super K> comparator()
```

This method returns the map's key comparator if it has been given one rather than relying on the natural ordering of the keys.

Getting the head and tail elements:

```
K firstKey()
K lastKey()
```

If the set is empty, these operations throw a `NoSuchElementException`.

Finding subsequences:

```
SortedMap<K,V> subMap( K fromKey, K toKey )
SortedMap<K,V> headMap( K toKey )
SortedMap<K,V> tailMap( K fromKey )
```

These operations work like the corresponding operations in `SortedSet`: the arguments do not themselves have to be actual keys, and the sets returned include the `fromKey` if it is present in the map, and do not include the `toKey`.

To see how a `SortedMap` could be useful, consider the problem we looked at earlier, of processing all the tasks in the system, in descending order of priority. This was straightforward using a list of tasklists, because lists retain their ordering:

```
for( List<Task> taskList : taskLists ) {
    for( Task t : taskList ) { ... }
}
```

but using an ordinary map we would have to get the ordering from the enum in which the priorities were defined:

```
for( Priority p : Priority.values() ) {
    List<task> taskList = taskMap.get( p );
    for( Task t : taskList ) { ... }
}
```

`SortedMap` can improve on this. We could use the ordering of the enum to construct it:

```
SortedMap<Priority,List<Task>> taskMap = new TreeMap<Priority,List<Task>>();
for( Priority p : Priority.values() ) {
    taskMap.put( p, new ArrayList<Task>() );
}
```

and then for the processing we could work through it using the same ordering, which it retains:

```
for( List<Task> taskList : taskMap.keySet() ) {
    for( Task t : taskList ) { ... }
}
```


7.3.1 TreeMap

`SortedMap` is implemented in the Collections Framework by `TreeMap`. We have already met trees as a data structure for storing elements in order when we discussed `TreeSet` (Section 4.2.1). In fact `TreeSet` is just a `TreeMap` with a standard value for all keys, so the explanation of the mechanism and performance of red-black trees given in Section 4.2.1 applies equally here.

The constructors for `TreeMap` include, beside the standard ones, one which allows you to supply a `Comparator` and one which allows you to create one from another `SortedMap`, using both the same comparator and the same mappings.

7.4 ConcurrentMap

Maps are often used in high-performance server applications, for example as cache implementations. Such applications are typically multi-threaded, to take advantage of multiple-CPU architectures, so there is a need for thread-safe `Map` implementations. In one sense this is quite easy to achieve; we will see in the next section that the Collections Framework provides a mechanism for wrapping a collection in a thread-safe wrapper so that only one thread at a time may operate on it. The drawback to this for an application running many threads is that the synchronized map becomes a bottleneck for which the threads have to queue, so removing one of the main advantages of having multiple threads. And even that may not be enough: suppose that an application wants to avoid ever overwriting an existing value in the map. Simply writing

```
if ( ! map.containsKey( key )) {    //1
    map.put( key, value );          //2
}
```

won't always work even if the map is synchronized, because immediately after line //1 is executed a different thread might intervene to `put` the key, and now our thread will go on in line //2 to overwrite the value that it has placed there. This is an example of conditional thread-safety, discussed in Section 2.6.

The interface `ConcurrentMap` addresses this problem by providing declarations for methods which are only conditionally thread-safe in the synchronized versions of ordinary `Map` implementations. There are four of these methods, which must perform atomically in `ConcurrentMap` implementations:

```
V putIfAbsent(K key, V value)
    // associate key with value only if key is not currently present
boolean remove(Object key, Object value)
    // remove key only if it is currently mapped to value.
V replace(K key, V value)
    // replace entry for key only if it is currently present
boolean replace(K key, V oldValue, V newValue)
    // Replace entry for key only if it is currently mapped to oldValue
```

7.4.1 ConcurrentHashMap

`ConcurrentHashMap` provides an implementation of `ConcurrentMap` and also offers a solution to the problem of reconciling throughput with thread safety. Retrievals usually do not block, even while the table is being updated (to allow for this, the contract states that the results of retrievals will reflect the most recently completed update operations **holding upon their onset**). Updates also can often proceed without blocking, because a `ConcurrentHashMap` consists of not one but a set of tables, called *segments*, each of which can be independently locked. If the number of segments is large enough relative to the number of threads accessing the table, there will be often be no more than one update in progress per segment at any time. It is never necessary—or even possible, for the client programmer—to lock the table as a whole.

`ConcurrentHashMap` is a useful implementation of `Map` in any application where it is not necessary to lock the entire table. But it is really indispensable in highly concurrent contexts, where it performs far better than a synchronized `HashMap`.

7.5 Comparison of Map Implementations

	add	contains	next	notes
<code>HashMap</code>	$O()$	$O()$	$O()$	
<code>TreeMap</code>	$O()$	$O()$	$O()$	
<code>LinkedHashMap</code>	$O()$	$O()$	$O()$	
<code>IdentityHashMap</code>	$O()$	$O()$	$O()$	

Chapter 8

Special Implementations

8.1 Pre-populated Collections

The `Collections` class provides convenient ways of creating some kinds of collections prepopulated with zero or more references to the same object. The simplest possible collections are empty:

```
static <T> List<T> emptyList()    // Returns the empty list (immutable)
static <K,V> Map<K,V> emptyMap()  // Returns the empty map (immutable)
static <T> Set<T> emptySet()      // Returns the empty set (immutable)
```

Empty collections can be useful in implementing methods to return collections of values; they can be used to signify that there were no values to return. If you have much occasion to use them, they can save you a good deal of unnecessary object creation. These methods replace the static `Collections` fields `EMPTY_SET`, `EMPTY_LIST`, and `EMPTY_MAP`, which were commonly used before Java 5 but are less useful now because their raw types generate unchecked warnings whenever they are used.

The `Collections` class also provides you with ways of creating collections objects containing only a single member:

```
static <T> Set<T> singleton(T o)
    // Returns an immutable set containing only the specified object.
static <T> List<T> singletonList(T o)
    // Returns an immutable list containing only the specified object.
static <K,V> Map<K,V> singletonMap(K key, V value)
    // Returns an immutable map, mapping only the key K to the value V.
```

Again these can be useful in providing a single input value to a method that is written to accept a Collection of values.

Finally, it is possible to create a list containing an arbitrary number of object references:

```
static <T> List<T> nCopies(int n, T o)
    // Returns an immutable list containing n references to the object o
```

Although the list produced by `nCopies` is itself immutable—in fact, as we showed in Section 6.3, it physically contains only a single element—the view that it provides can be used as the basis for building further collections, for example by providing it as the argument to a constructor or an `addAll` method.

8.2 Decorated Collections

The `Collections` class provides ways of modifying the behaviour of standard collections classes in three ways—by making them unmodifiable or dynamically type-safe, or by synchronizing them. It does this by creating specialised objects which mostly delegate their real work to the standard collection object, but modify the way in which it is done. This is an example of the use of the *decorator pattern* (Gamma et. al., *Design Patterns*), which provides an alternative to inheritance when that would result in an explosion of closely related subclasses. Implementing the decorator pattern normally involves creating a decorator class which implements the modifications needed to the behaviour of the collection; here, the decorator classes are hidden inner classes of the `Collections` class, which instead of exposing them provides factory methods to return instances of them.

8.2.1 Unmodifiable Collections

An unmodifiable collection will throw `UnsupportedOperationException` in response to any attempt to change its structure or the elements which compose it. This can be useful when you want to allow clients read access to an internal data structure. Passing the structure in an unmodifiable wrapper will prevent a client from changing it—provided, of course, that it does not contain references to modifiable objects. (If that is in fact the case, you may have to protect your internal data structure by providing clients instead with a *defensive copy* made for the purpose).

There are six unmodifiable wrapper factory methods, corresponding to the six major interfaces of the Collections Framework:

```
public static <T> Collection<T>
    unmodifiableCollection(Collection<? extends T> c);
public static <T> Set<T> unmodifiableSet(Set<? extends T> s);
public static <T> List<T>
    unmodifiableList(List<? extends T> list);
public static <K, V> Map<K, V>
    unmodifiableMap(Map<? extends K, ? extends V> m);
public static <T> SortedSet<T>
    unmodifiableSortedSet(SortedSet<? extends T> s);
public static <K, V> SortedMap<K, V>
    unmodifiableSortedMap(SortedMap<K, ? extends V> m);
```

8.2.2 Synchronized Collections

As we explained in Section 2.6, most of the Framework classes are not thread-safe—by design—in order to avoid the overhead of unnecessary synchronization as incurred by the legacy classes `Vector` and `Hashtable`. But there are occasions when you do need to program multiple threads to have access to the same collection, and these synchronized wrappers are provided by the `Collections` class for such situations. Again there is a factory method for each of the major Framework interfaces:

```
public static <T> Collection<T>
    synchronizedCollection(Collection<T> c);
public static <T> Set<T> synchronizedSet(Set<T> s);
public static <T> List<T> synchronizedList(List<T> list);
public static <K, V> Map<K, V> synchronizedMap(Map<K, V> m);
public static <T> SortedSet<T>
    synchronizedSortedSet(SortedSet<T> s);
public static <K, V> SortedMap<K, V>
    synchronizedSortedMap(SortedMap<K, V> m);
```

The decorator classes that provide these synchronized views are conditionally thread-safe (Section 2.6); although each of their operations is guaranteed to be atomic, you may need to synchronize multiple method calls in order to obtain consistent behaviour. In particular, iterators must be created and used entirely within a code block synchronized on the collection, otherwise the result will at best be failure with `ConcurrentModificationException`. This is very coarse-grained synchronization; if your application makes heavy use of synchronized collections, its effective concurrency will be greatly reduced.

8.2.3 Checked Collections

Unchecked warnings are a signal to us to take special care to avoid run-time type violations. But it may be beyond our power to ensure that they never occur; for example, after we have passed a typed collection reference to an ungenerified library method, we can't be sure that it has added only correctly typed elements to the collection. Rather than lose confidence in the collection's type safety we can pass in a checked wrapper, which will explicitly cast every element added to the collection into the type supplied when it is created. Checked wrappers are supplied for the main interfaces:

```
public static <E> Collection
    checkedCollection(Collection<E> c, Class<E> elementType)
public static <E> List
    checkedList(List<E> c, Class<E> elementType)
public static <E> Set
    checkedSet(Set<E> c, Class<E> elementType)
public static <E> SortedSet
```

```

        checkedSortedSet(SortedSet<E> c, Class<E> elementType)
    public static <K, V> Map
        checkedMap(Map<K, V> c, Class<K> keyType, Class<V> valueType)
    public static <K, V> SortedMap
        checkedSortedMap(SortedMap<K, V> c, Class<K> keyType, Class<V> valueType)

```

A related use of the checked wrappers is to ensure that a client has not smuggled a wrongly typed element into a typed collection you are being supplied with. Suppose, for example, a library class receives a collection of objects representing secure channels, which it will use for transmitting information. To ensure that every element really has the right type, it could create a new empty checked collection, and add all the elements of the suspect collection to it.

```

    public void transmitSensitiveInformation( List<SecureChannel> list ) {
        List<SecureChannel> secureList =
            Collections.checkedList( new ArrayList<SecureChannel>(), SecureChannel.class );
        secureList.addAll( list );
        ...
        // from now on use only secureList
    }

```

Like the other decorated collections, checked wrappers have fairly severe limitations. Since the check takes place at runtime, only runtime type information can be used, so the only types that can be fully checked are reifiable ones. [\[Insert reference to reification explanation in Part I and reiterate definition if necessary\]](#) By contrast, the best that you can do by way of run-time type checking for a collection of elements of a parameterised type is to create a wrapper that will check that each added element belongs to the corresponding raw type. (This partial solution is made even more unsatisfactory by the fact that the checked wrapper will itself be a collection of the raw type, so you won't be able to use it without getting unchecked warnings).

Chapter 9

Generic Algorithms

The Collections Framework supplies a useful set of generic algorithms in the class `Collections`. Generic algorithms represent reusable functionality; they can be applied to `Lists` (or in some cases to `Collections`) of any type. Generifying the types of these methods has led to some fairly complicated declarations, so each section discusses the declarations briefly after presenting them. ??? gives a fuller motivation for these and other design decisions taken in generifying the Collections Framework.

The generic algorithms fall into four major categories: changing element order in lists, finding extreme values in a collection, searching a list, and systematically changing the contents of a list.

9.1 Changing the Order of List Elements

`swap` is the simplest of these methods; it executes in constant time. `sort` is the most complex; it transfers the elements into an array, applies a merge sort to them in time $O(n \log n)$, and then returns them to the `List`. The remaining methods all execute in time $O(n)$, although the coefficient will depend on whether the `List` should be processed using sequential or random access. Each method selects between two algorithms based both on whether the `List` implements the interface `RandomAccess`, and also on its size—for small sequential access lists the random access algorithms are faster.

```
static void reverse(List<?> list)
    // reverses the order of the elements
static void rotate(List<?> list, int distance)
    // rotates the elements of the list; the element at index
    // i is moved to index (distance + i) % list.size()
static void shuffle(List<?> list)
    // randomly permutes the list elements
static void shuffle(List<?> list, Random rnd)
```

```

        // randomly permutes the list using the randomness source rnd
static <T extends Comparable<? super T>> void sort(List<T> list)
        // sorts the supplied list using natural ordering
static <T> void sort(List<T> list, Comparator<? super T> c)
        // sorts the supplied list T using the supplied ordering
static void swap(List<?> list, int i, int j)
        // swaps the elements at the specified positions

```

9.2 Finding Extreme Values in a Collection

`min` and `max` are supplied for this purpose, each with two overloads—one using natural ordering of the elements, and one accepting a `Comparator` to impose an ordering. They execute in $O(n)$ time.

```

static <T extends Object & Comparable<? super T>> T
    max(Collection<? extends T> coll)
        // returns the maximum element using natural ordering
static <T> T max(Collection<? extends T> coll, Comparator<? super T> comp)
        // returns the maximum element using the supplied comparator
static <T extends Object & Comparable<? super T>> T
    min(Collection<? extends T> coll)
        // returns the minimum element using natural ordering
static <T> T min(Collection<? extends T> coll, Comparator<? super T> comp)
        // returns the maximum element using the supplied comparator

```

Why are these declarations so unwieldy? In the case of the overloads that use natural ordering, the reason for the complexity of the return type lies in the need to maintain compatibility with code that makes calls to the older, non-generified, version. The older version returns `Object`, so that has to be the return type of the erasure of the generified version too.

In the case of the overloads that accept a `Comparator`, the types are chosen to make it clear that the method can be applied to any `Collection-Comparator` pair where the type parameter of the `Collection` is a subtype of the type parameter of the `Comparator` (see ???).

9.3 Finding Specific Values in a List

Methods in this group locate elements or groups of elements in a `List`, again choosing between alternative algorithms on the basis of the list's size and whether it implements `RandomAccess`. [Explain binary search, give O for sequential access list.]

```

static <T> int binarySearch(List<? extends Comparable<? super T>> list, T key)
        // searches for key using binary search
static <T> int binarySearch(List<? extends T> list, T key, Comparator<? super T> c)
        // searches for key using binary search

```



```
static int indexOfSubList(List<?> source, List<?> target)
    // finds the first sublist of source which matches target
static int lastIndexOfSubList(List<?> source, List<?> target)
    // finds the last sublist of source which matches target
```

The signature of the first `binarySearch` overload says that you can use it to search for a key of type `T` in a list of objects which can have any type—provided they can be compared with objects of type `T` or a supertype of `T`. The second is like the `Comparator` overloads of `min` and `max`, except that in this case the type parameter of the `Collection` must be a subtype of the type of the key, which in turn must be a subtype of the type parameter of the `Comparator`.

The signatures of `indexOfSubList` and `lastIndexOfSubList` allow the source and target lists to be of any kind (remember that the two wildcards signify unrelated types). No assumptions need to be made about the types of the list elements; they are simply going to be compared using `equals`.

9.4 Changing the Contents of a List

These methods change some or all of the elements of a list. `copy` transfers elements from the source list into an initial sublist of the destination list (which has to be long enough to accomodate them), leaving any remaining elements of the destination list unchanged. `replaceAll` replaces every occurrence of one value in a list with another—where either the old or new value can be null—returning `true` if any replacements were made. `fill` replaces every element of a list with a specified object.

```
static <T> void copy(List<? super T> dest, List<? extends T> src)
    // copies all of the elements from one list into another
static <T> void fill(List<? super T> list, T obj)
    // replaces every element of list with obj
static <T> boolean replaceAll(List<T> list, T oldVal, T newVal)
    // replaces all occurrences of oldVal in list with newVal
```

The signatures of these methods can all be explained using the Get and Put Principle. `copy` was discussed in Part I. It gets elements from the source list and put them into the destination, so the types of these lists are respectively `? extends T` and `? super T`. This fits with the intuition that the type of the source list elements should be a subtype of the destination list. For `fill`, the Get and Put Principle dictates that you should use `extends` if you are putting values in to a parameterised collection, and for `replaceAll`, it states that if you are putting values into and getting values out of the same structure you should not use wildcards at all.